AI 500
Foundations of Artificial Intelligence

L1: Introduction

By Murtaza Taj

LUMS
A Not-for-Profit University

Team

Foundations of AI AI500

Murtaza Taj
murtaza.taj

Mon 15:30 - 16:30
Tues 14:30 - 15:40

Haider Dawood Khan
26100053

Wed 11:00 - 12:00
Sat 12:00-13:00

Eeman Adnan
24280022

Tues 17:00 - 18:00
Sat 12:30 - 13:30

Muhammad Ashhad Ali
26100023

Thurs/Sat
17:00-18:00

Sarim Malik
(Head TA)
26100129

Mon/Wed
14:00-15:00
Sun 11:00 - 12:00

Dr. Murtaza Taj

- Murtaza Taj (PhD)
  - PhD from Queen Mary University of London in Dec 2009
  - Joined LUMS in Jan 2011
  - Director Computer Vision & Graphics Lab
  - Research: 2D & 3D Scene Understanding, Image Processing, Computer Graphics, Machine Learning

TPI Technology for People Initiative

tpi.lums.edu.pk

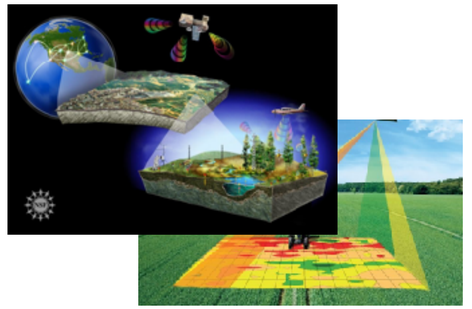
CV Lab LUMS

cvlab.lums.edu.pk

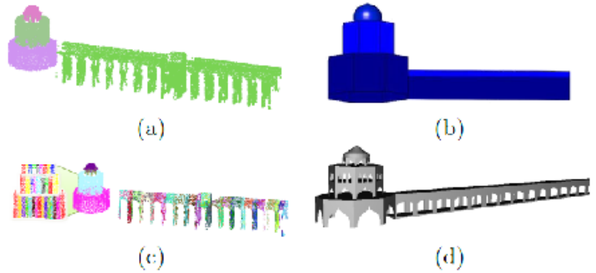
NCRA NATIONAL CENTRE OF ROBOTICS & AUTOMATION | Agricultural Robotics Lab

Computer Vision & Graphics Lab (CVG Lab)

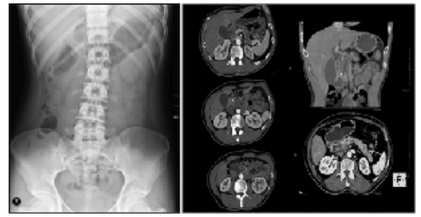
https://cvlab.lums.edu.pk



- Remote Sensing
  - Image matching
  - Retrieval, classification, object detection



- Digital Cultural Heritage (https://sketchfab.com/cvlablums)
  - 2D/3D Scene Understanding
  - 3D Object Retrieval, Point cloud segmentation



- Medical Imaging
  - Generative Adversarial Networks
  - Image classification

Murtaza Taj

Co-founder Groopic Inc. CA, 2012 - 2015

c|net

Google for Entrepreneurs

CNN

TechCrunch

TECHJUICE

TECHINASIA

SAMAA

FOX NEWS

What is the core business of Google & Facebook?

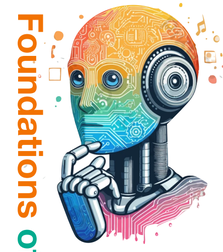
Murtaza Taj

Co-founder Ingrain Media Inc. CA (2013 - 2016)

INGRAIN

Ingrain is the smartest, fastest & the most efficient way of integrating brands in videos.

Research Planning Advice



- Fall 2025
- **Spring 2026**
  - Choose elective with instruction who you want to be your research/MS Thesis advisor
- **Summer 2026**
  - Start your research here
- Fall 2026
- Spring 2027
- Faculty/Student Ratio

AI 500
Foundations of Artificial Intelligence

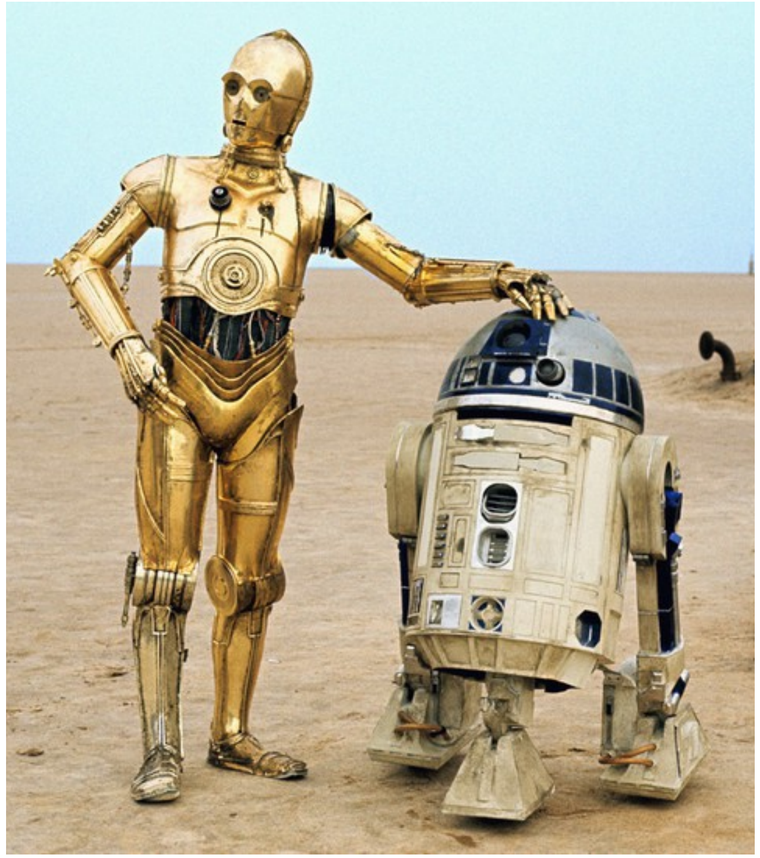
By Murtaza Taj

LUMS
A Not-for-Profit University

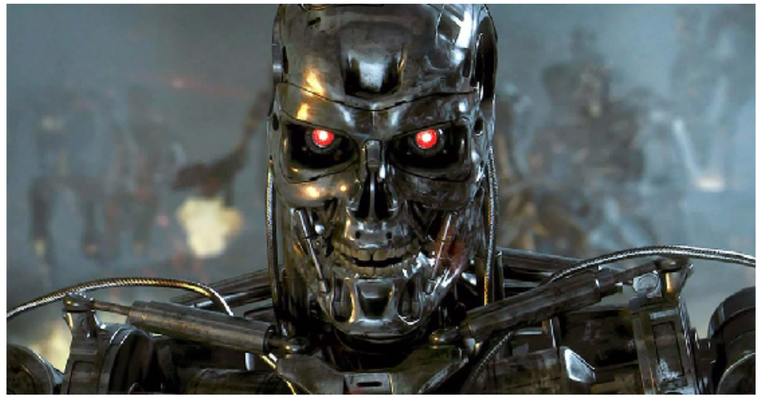
Today

- What is AI and what it can do?
- What is this course about?
- Introduction to Agent, Environment, Search

AI in Science Fiction



AI in 70’s
C-3PO - Translation
R2-D2 - bit of everything

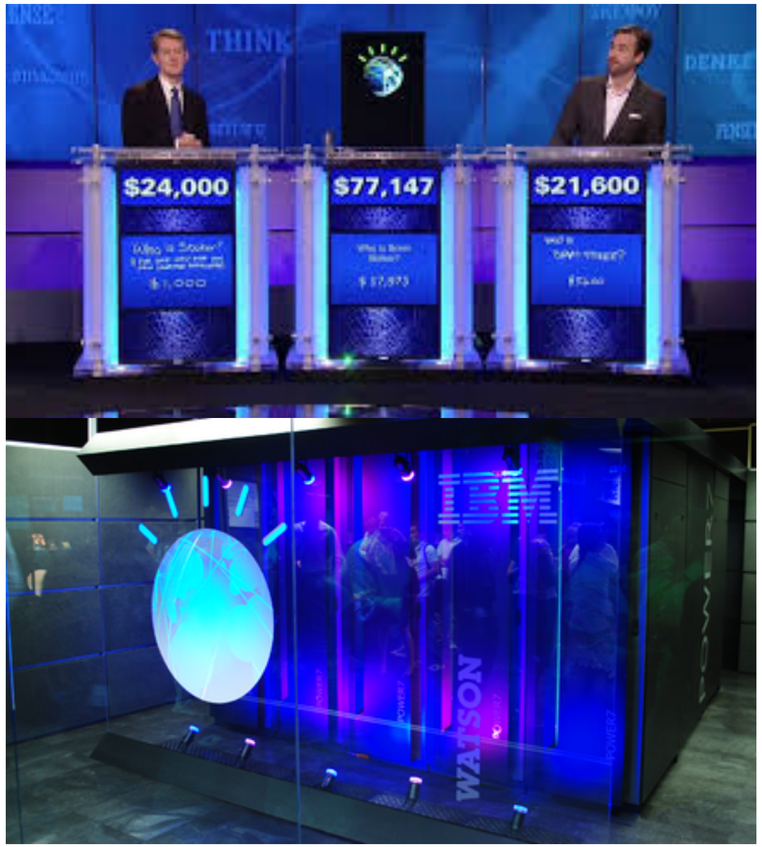


AI in 80’s
Terminator - Killer robot from future
C3PO with different software Matrix

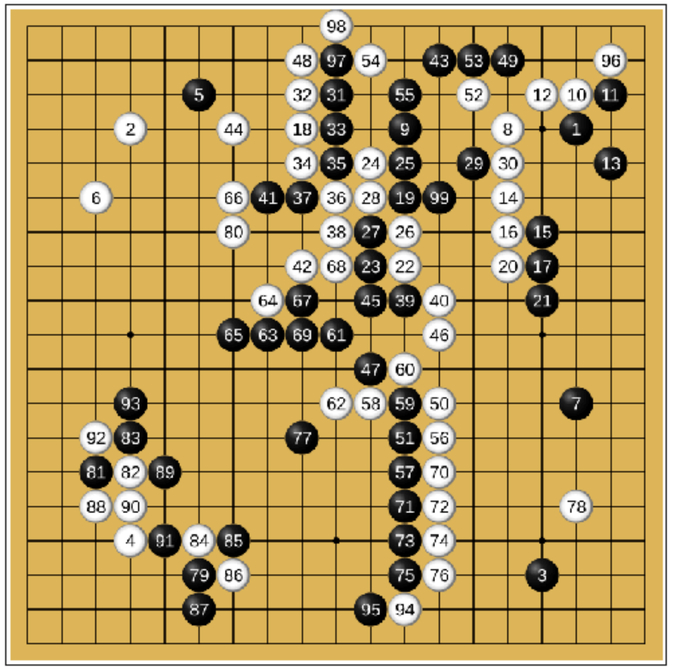
AI in 2000’s
Galactica/Cylons - Indistinguishable Robots
Westworld

Image credits: Google Images

AI in News



2011
IBM Watson Q&A
Defeated Humans in
Jeopardy



AlphaGo

2016
AlphaGo by DeepMind

2017
AI in Health care and LLMs

Image credits: Google Images

Course Outline

Part 1: AI Agents, Search, Classical ML

Week 1: Introduction

- Course Overview: Objectives, structure, and expectation
- Introduction to AI and its applications
- Introduction to Search Algorithms

Week 2: Search

- Formalizing actions as a function
- Transition models and their importance
- A* search
- Minimax, Alpha/Beta pruning
- Python refresher
- **Programming Assignment 1: Search**

Week 3-4: Un/supervised Learning and Decision Trees

- Supervised vs. Unsupervised learning
- K-means Clustering
- Agglomerative Clustering
- Decision Trees
- Bagging
- Random Forest and Boosting
- **Programming Assignment 2: Unsupervised learning**

Week 5: Support Vector Machine

- Formulation: Hard SVM and Soft SVM
- Optimization problem formulation (including the concept of duality in optimization)
- Applications and kernel trick

Course Outline

Part 2: Fundamentals of Data Science

Week 6: Data Science Life Cycle

- Course Overview: Objectives, structure, and expectation
- Introduction to AI and its applications
- What and Why of Data Science
- Data Science Life Cycle
- Intro to Pandas/Python

Week 7: Data Cleaning, Exploratory Data Analysis and Visualization

- Data Sampling (Random, Cluster, Stratified)
- Data Cleaning and Exploratory Data Analysis
- Data Visualization and Transformations
- Case Studies using datasets like crime, election, landslide etc via Pandas/Python
- **Programming Assignment 3**

Week 8: Oct 25th 2025

Week 9-11: Introduction to Causality

- Correlation and causation
- Causal questions: "What if?", "Why?", "How?" Counterfactual thinking
- Real-world problems where causal reasoning matters (e.g., medicine, economics, policy)
- Predictive vs. Causal Questions
- Randomised Control Trials
- Graphical Models & Structural Causal Models (SCMs)
- DAGs as reasoning tools
- Drawing Causal Diagrams (DAGs)
- Confounders, Colliders, Mediators
- **Programming Assignment 4**

Course Outline

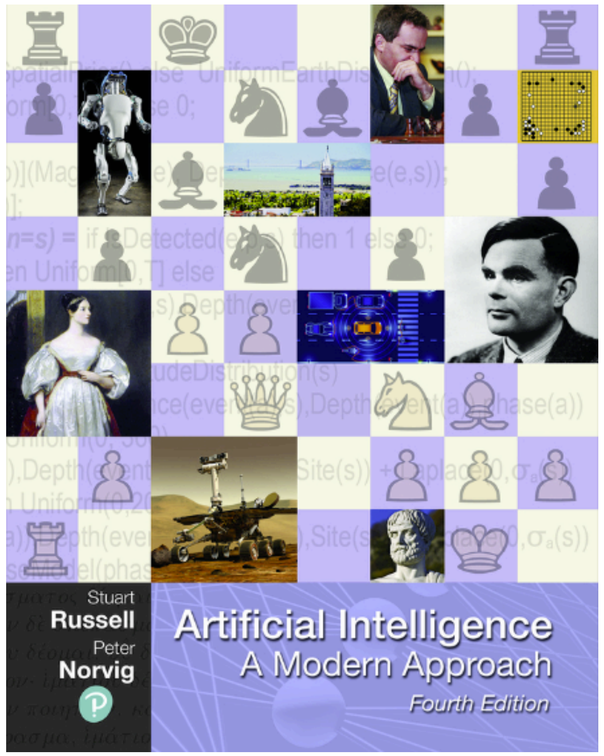
Part 2: Neural Networks

Week 12-13: Neural Networks and Back Propagation

- Linear Regression
- Perceptron Model: Binary classifiers, activation functions
- Multilayer Networks: Architecture, forward and backward propagation
- Loss Functions: Cross-entropy, mean squared error
- Training and Tuning: Hyperparameter tuning, learning rates
- Optimization and learning using back propagation
- Introduction to tensor flow and/or pytorch
- **Programming Assignment 5: Neural Network**

Week 14: Advanced Neural Networks and Deep Learning

- Deep Learning Concepts: Deep networks (Filters, convolution, filter-banks)
- Convolutional neural networks
- Examples with tensor flow and/or pytorch - Image classification via CNN



Artificial Intelligence: A Modern Approach by Stuart Russell and Peter Norvig, 4th Edition

Course Components

- Assignments, 35 % - in Python (on PC, Colab PyTorch, Tensor Flow)
- Quizzes, 15 % - Open book, open notes (handwritten), note we have 14 weeks
  (Note: Digital Notes will not be allowed during quizzes etc)
- Mid-Exam, 20 % - written
- Final Exam, 30 % - written + programming
- **Late submission Policy**
  10% marks deduction per day
  Zero on the 6th day
  No late submission after last day of classes

AI Policy

AI Policy:

Any character in a code/ assignment or any other graded instrument not written by you **must** be properly referenced.

Use of these tool must not cause obstruction in learning.

We will conduct vivas and automated checks to ensure you have learned what you have submitted.

Mixed Policy → Selective AI Utilization

This syllabus statement is useful when you are allowing the use of AI tools for certain purposes, but not for others. Refer to Appendix A for a visual guide to help instructors map AI usage for their assignments.

Template for Course Syllabus

The use of generative AI tools (e.g., ChatGPT, Gemini, Llama, Dall-e, etc.) is permitted in this course for the following activities:

- Brainstorming and refining your ideas,
- Fine-tuning your research questions,
- Finding information on your topic,
- Drafting an outline to organize your thoughts, and
- Checking grammar and style.

However, the use of generative AI tools is not permitted in this course for the following activities:

- Submitting AI-generated content as your own in classroom settings, such as discussion posts, assigned responses, or Zoom chat contributions.
- Completing group work that your group has assigned to you, unless it is mutually agreed upon that you may utilize the tool.
- Writing a draft of a writing assignment.
- Writing entire sentences, paragraphs, or papers to complete class assignments.

These activities are restricted because they undermine individual learning, critical thinking, and the development of essential skills, or misrepresent the student's own contributions to the work. Violations of this policy will be treated as academic misconduct. Please note that AI policies may vary across courses at LUMS, and it is the student's obligation to adhere to the specific requirements of each course.

Intelligence:

Capacity for learning, reasoning, understanding, problem solving

Artificial Intelligence:

Build a program/machine that appears intelligent to the user in some domain

General AI:

Build a system that’s good at everything

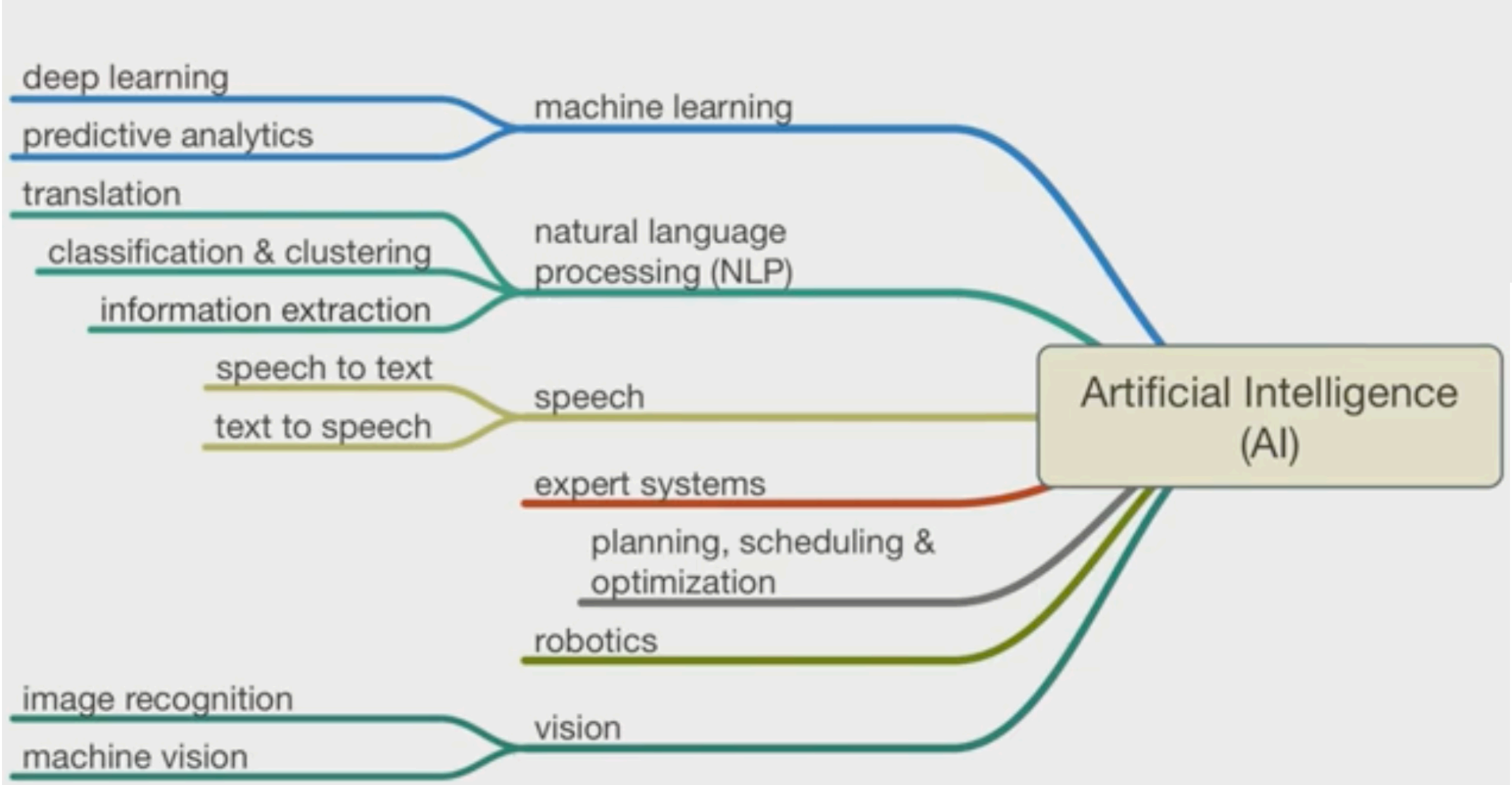
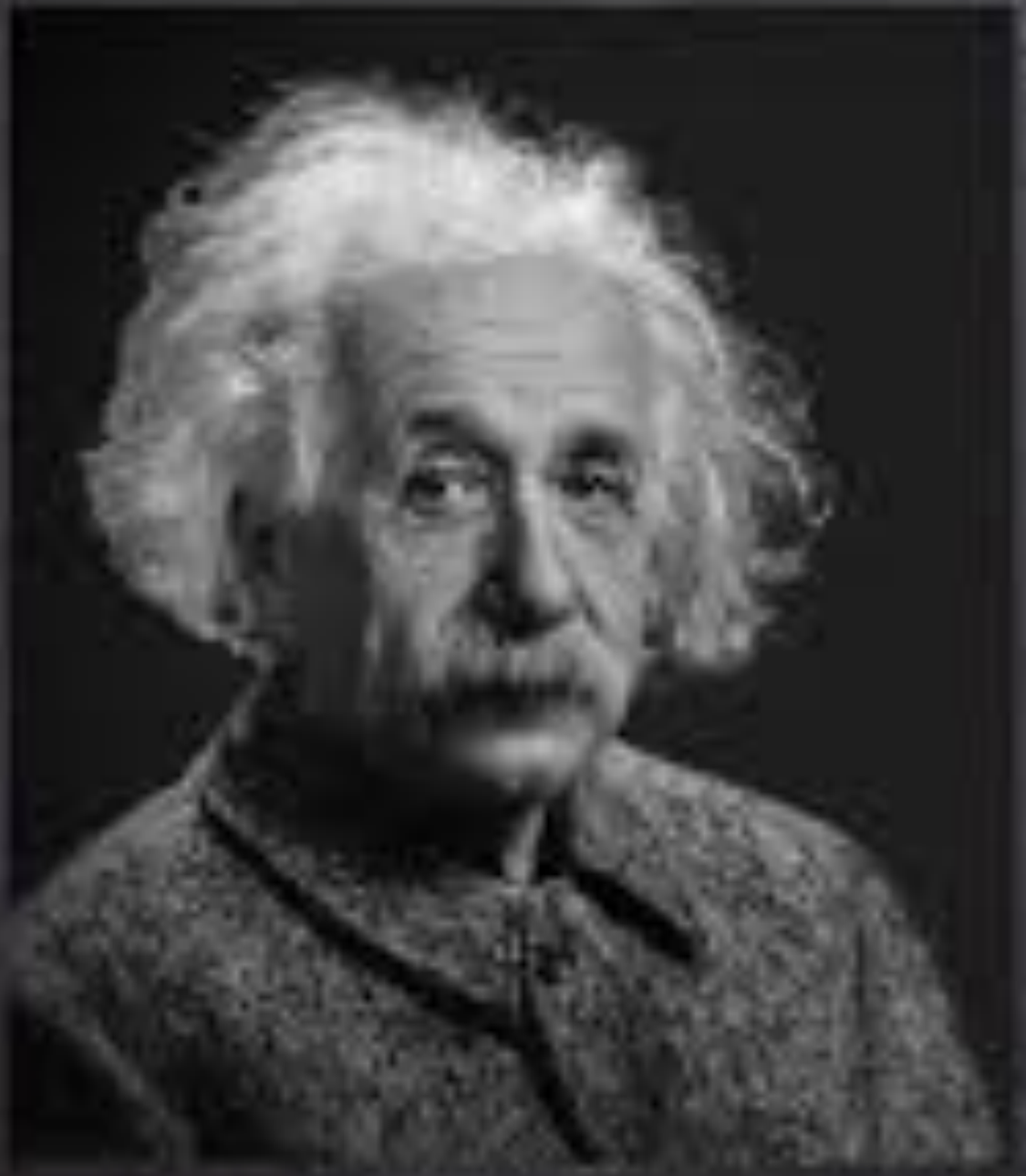


Image credits: Dave Churchill



"The important thing
is not to stop
questioning.
Curiosity has its
own reason
for existing."

– Albert Einstein

THE
IMITATION
GAME

THE FOLLOWING PREVIEW HAS BEEN APPROVED TO

ACCOMPANY THIS FEATURE

BY THE MOTION PICTURE ASSOCIATION OF AMERICA, INC.

www.filmratings.com

www.mpaa.org

Can Machine Think?

Alan Turing
1912 - 1954

Designed computing

Designed one of the first actual operational computers

Desgined computer that was able to help computer break codes in WWII - saved million of lives

But he was imprisoned by British Government

He saved the world but he killed himself

Turing test

1950 paper titled 1950 paper "Computing Machinery and Intelligence by Alan Turing

The Imitation Game:

Player A (Computer), Player B (Human)

the player C (interrogator), is given the task of trying to determine which player – A or B – is a computer and which is a human. The interrogator is limited to using the responses to written questions to make the determination.[1]

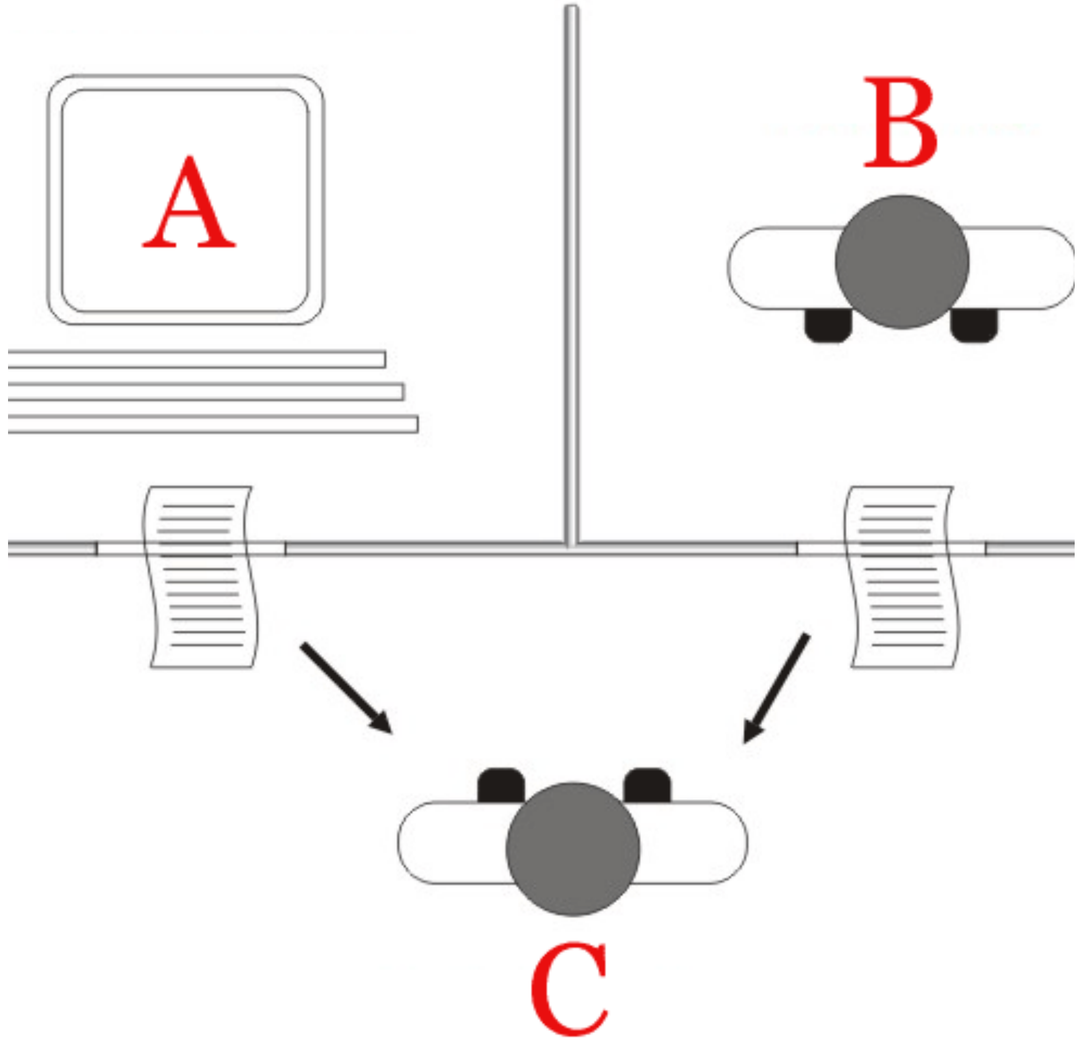


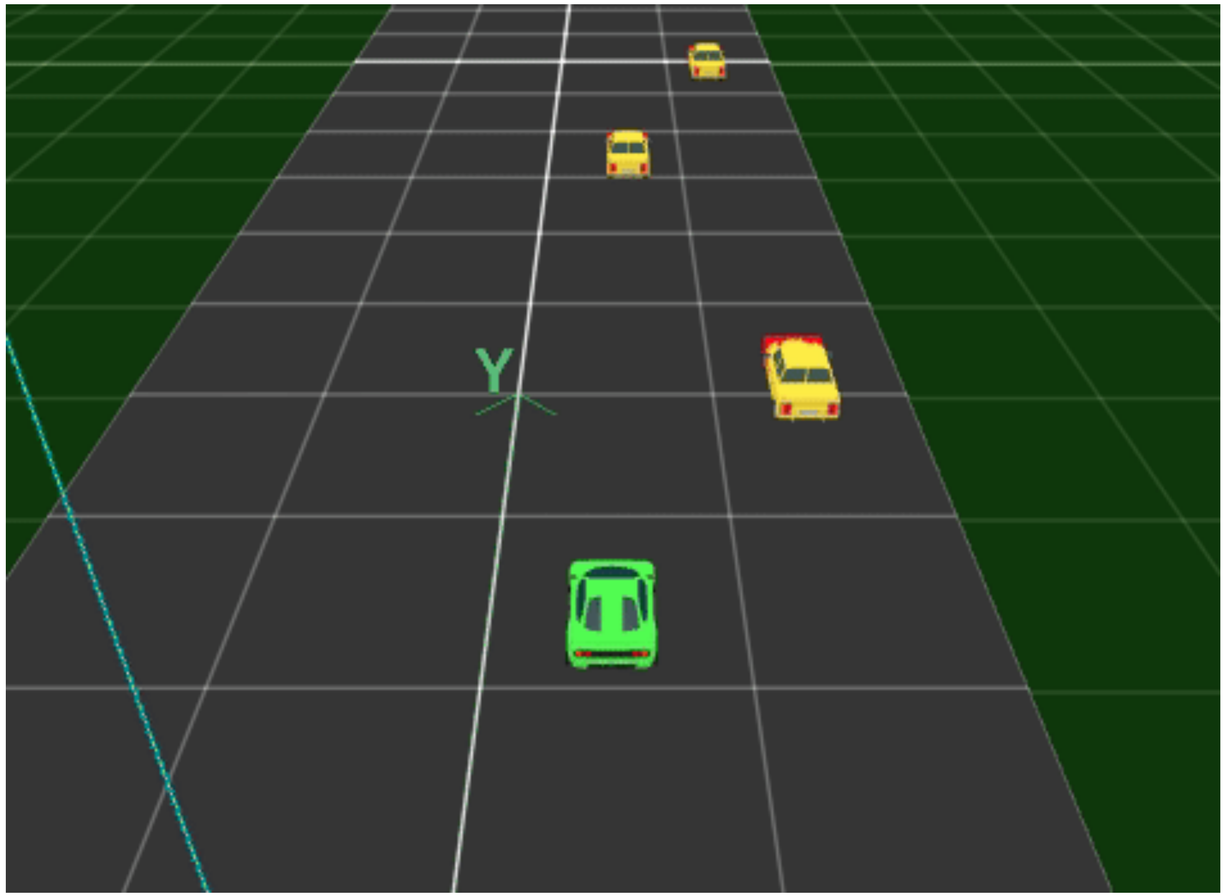
Image credits: Wikipedia

Playing game measure intelligence

What is a game?

- **Agent** in an **environment**
  - Agent can take **actions**
  - Actions **effects** the environment

- Agent has a **goal**
  - Move $\leftarrow \rightarrow \uparrow, \downarrow$
  - Defeat the oponent etc
  - Get most points



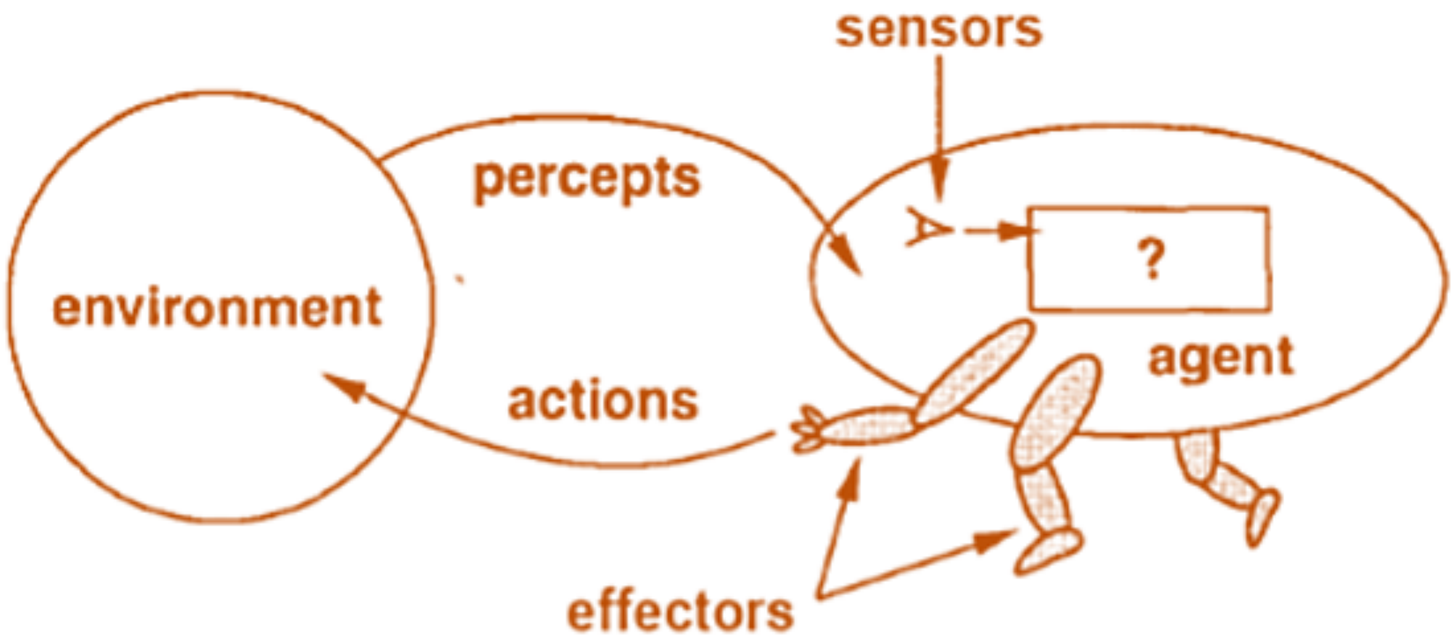
Agent

Image credits: Google Images

```python
import os
import pandas as pd
from pandasai import Agent

# Sample DataFrame
sales_by_country = pd.DataFrame({
    "country": ["United States", "United Kingdom", "France"],
    "revenue": [5000, 3200, 2900]
})

os.environ["PANDASAI_API_KEY"] = "YOUR_API_KEY"

agent = Agent(sales_by_country)
agent.chat('Which are the top 5 countries by sales?')
```

Image credits: Google Images

An **agent** is anything that can be viewed as *perceiving* its **environment** through **sensors** and *acting* upon that environment through **effectors.**



www.educba.com

Sensors

- Camera, Lidar, Radar

Percept

- Agent's current belief of the world,
- Environment's current **state** s

Actions

- Keyboard input
- Action a taken by an agent in a given state **transitions** from the state to another
- State transition function
- Table, Graph, function
- $s' = f(s, a)$

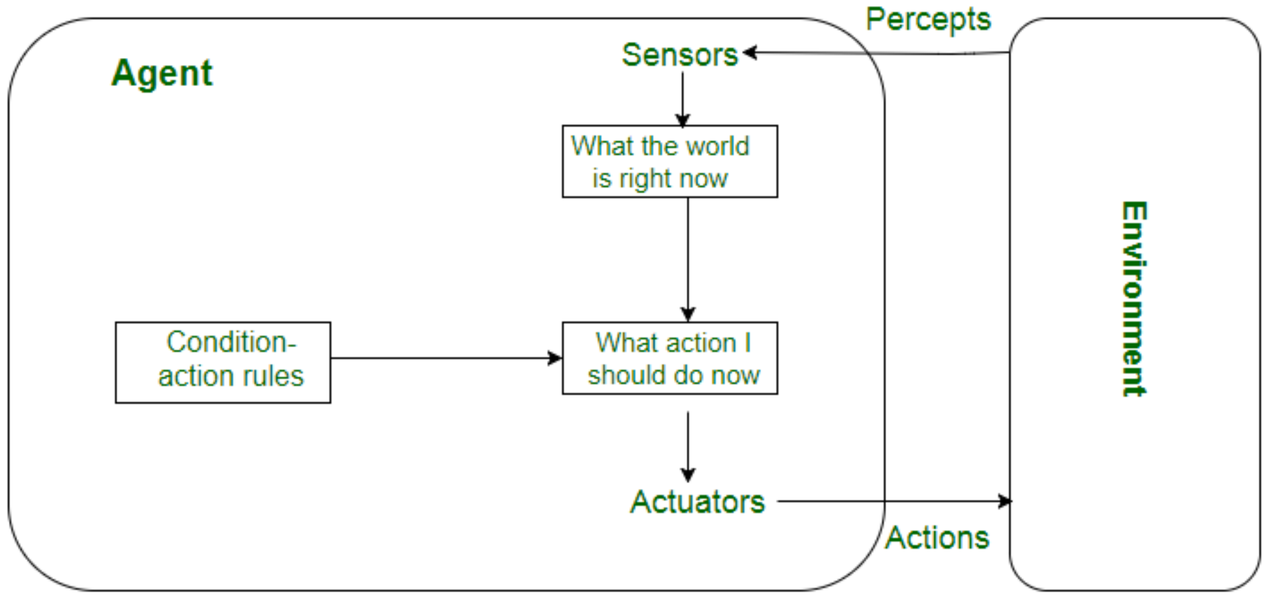
Policy

- Policy maps from state to action
- Policy π = Optimal Policy π^*
- $\pi(s) = a$ Given we are in state s, do action a
- Policy could be implemented as:
  - Lookup Table
  - If-statements
  - Neural Network etc

Simple Reflex Agent

- Ignores percept histroy
- Based on condition-action rules

  (Maps states to actions)
- Very limited intelligence
- If environment changes, the rules need to be updated



Example Policy: Grid Path Finding

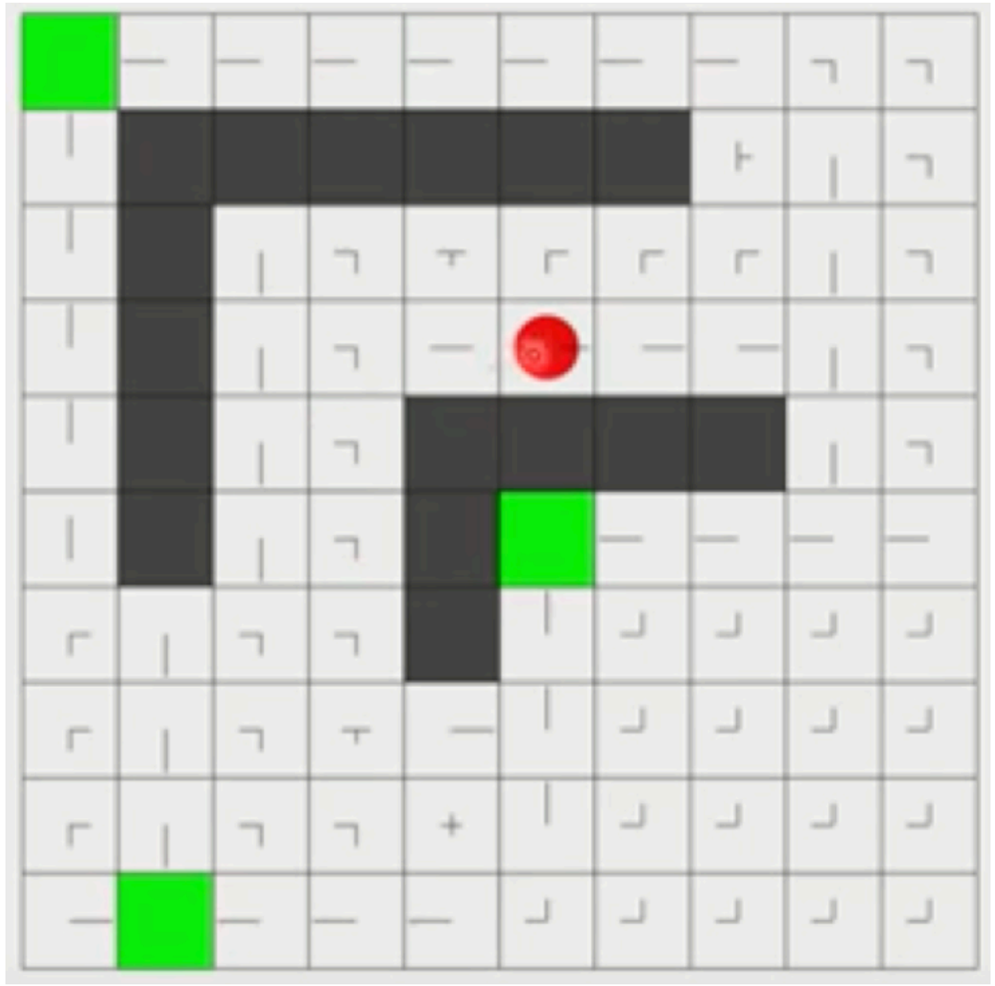


Image credits: Dave Churchill

Foundations of AI
AI500

Policy: Black Jack

Hard Hands										
	Dealer's Up Card									
Your Hand	2	3	4	5	6	7	8	9	10	A
17+	ST	ST	ST	ST	ST	ST	ST	ST	ST	ST
16	ST	ST	ST	ST	ST	H	H	H	H	H
15	ST	ST	ST	ST	ST	H	H	H	H	H
14	ST	ST	ST	ST	ST	H	H	H	H	H
13	ST	ST	ST	ST	ST	H	H	H	H	H
12	H	H	ST	ST	ST	H	H	H	H	H
11	DD	DD	DD	DD	DD	DD	DD	DD	DD	H
10	DD	DD	DD	DD	DD	DD	DD	DD	H	H
9	H	DD	DD	DD	DD	H	H	H	H	H
5-8	H	H	H	H	H	H	H	H	H	H

Soft Hands										
	Dealer's Up Card									
Your Hand	2	3	4	5	6	7	8	9	10	A
A 8-10	ST	ST	ST	ST	ST	ST	ST	ST	ST	ST
A, 7	ST	DD	DD	DD	DD	ST	ST	H	H	H
A, 6	H	DD	DD	DD	DD	H	H	H	H	H
A, 5	H	H	DD	DD	DD	H	H	H	H	H
A, 4	H	H	DD	DD	DD	H	H	H	H	H
A, 3	H	H	H	DD	DD	H	H	H	H	H
A, 2	H	H	H	DD	DD	H	H	H	H	H

SP = SPLIT
ST = STAND
H = HOLD
DD = DOUBLE DOWN

Basic Blackjack Strategy Charts

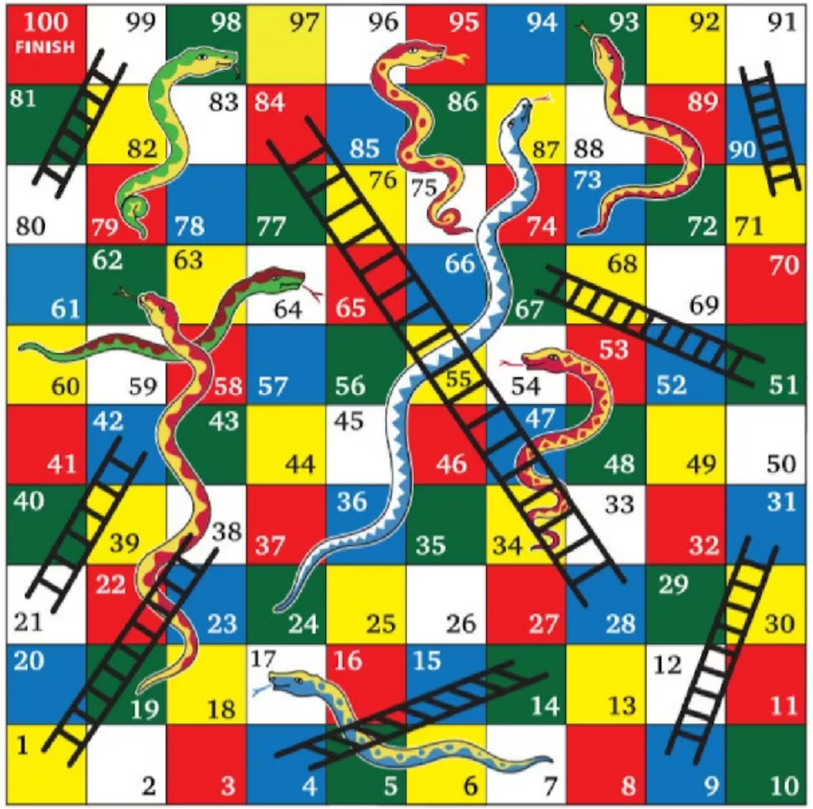
What you do with your hand might change based on the variation of blackjack you're playing, as well as how many decks are being used. **These charts are best suited for single-deck blackjack**

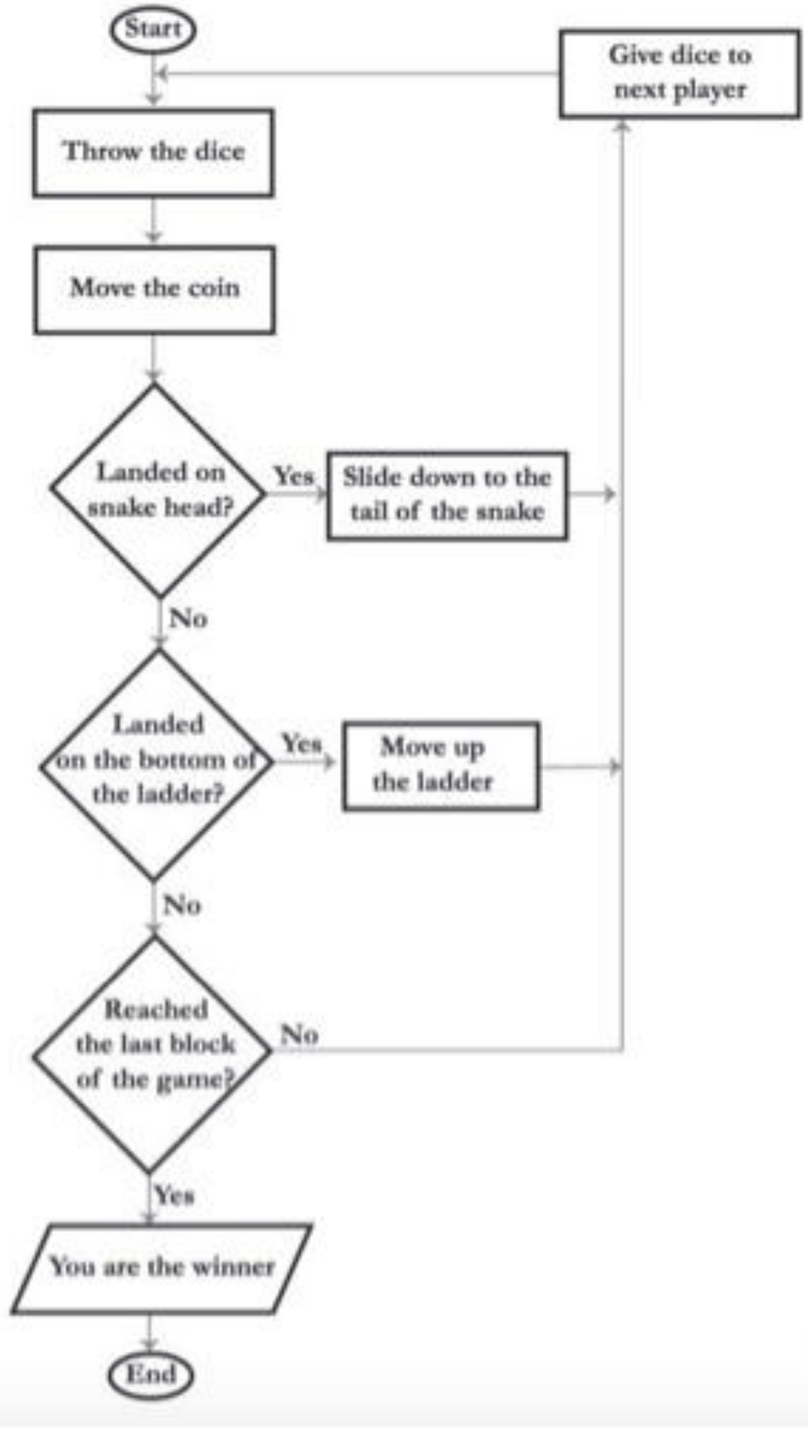
Purchasing Insurance is NOT recommended

Pairs										
	Dealer's Up Card									
Your Hand	2	3	4	5	6	7	8	9	10	A
A, A	SP	SP	SP	SP	SP	SP	SP	SP	SP	SP
10, 10	ST	ST	ST	ST	ST	ST	ST	ST	ST	ST
9, 9	SP	SP	SP	SP	SP	ST	SP	SP	ST	ST
8, 8	SP	SP	SP	SP	SP	SP	SP	SP	SP	SP
7, 7	SP	SP	SP	SP	SP	SP	H	H	H	H
6, 6	SP	SP	SP	SP	SP	H	H	H	H	H
5, 5	DD	DD	DD	DD	DD	DD	DD	DD	H	H
4, 4	H	H	H	SP	SP	H	H	H	H	H
3, 3	SP	SP	SP	SP	SP	SP	H	H	H	H
2, 2	SP	SP	SP	SP	SP	SP	H	H	H	H

Example Policy

Sankes & Ladders





Foundations of AI

AI500

State / Action Space

Used as an estiamtion of complexity

- State Space
  - Number of possible configuration of the environment for a given problem

- Action Space
  - Number of actions possible from a state

Example: Tic-Tac-Toe

State-Space Complexity Calculation

X
O
[blank]

$$3^9$$

$$3^9 = 19{,}863$$

1	2	3
4	5	6
7	8	9

Example: Chess

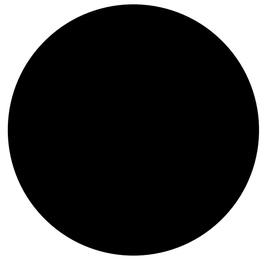
State-Space Complexity Calculation

- 64 Space
- 32 Empty Spaces
- 8 Pawns in 2 sets
- 2 Pieces in 6 sets (Rook, Night, Bishop)

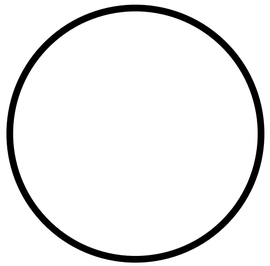
$$\frac{64!}{32!8!8!2!2!2!2!2!2!} = 4.6 \times 10^{42}$$

Example: GO

State-Space Complexity Calculation



$$3^{19\times19}$$



$$3^{361}$$

[blanks]

$$1.7\times10^{172}$$

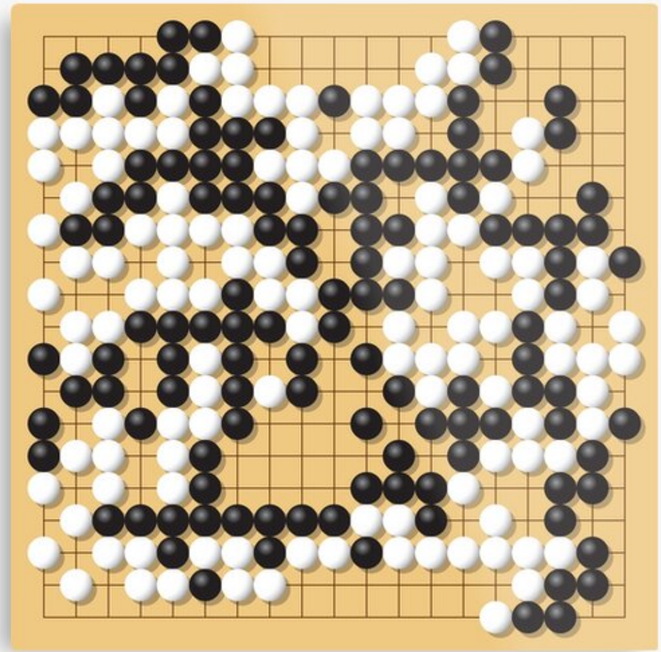
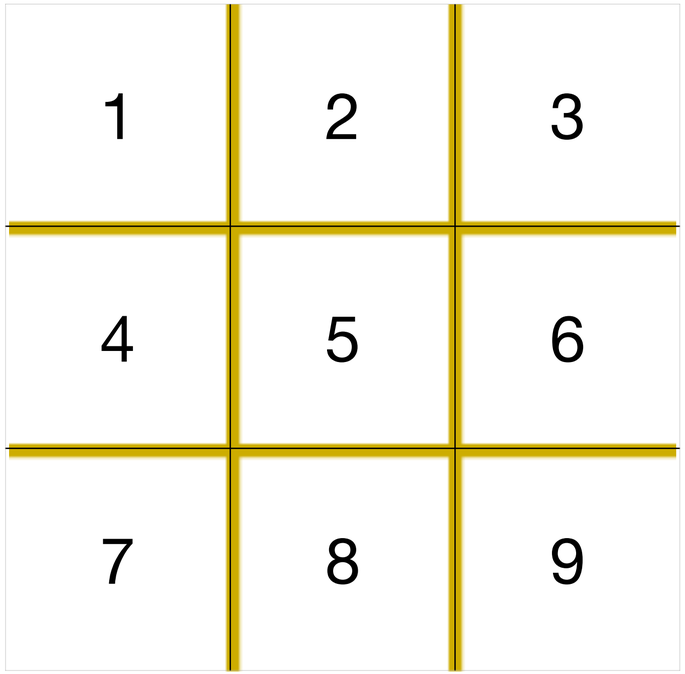


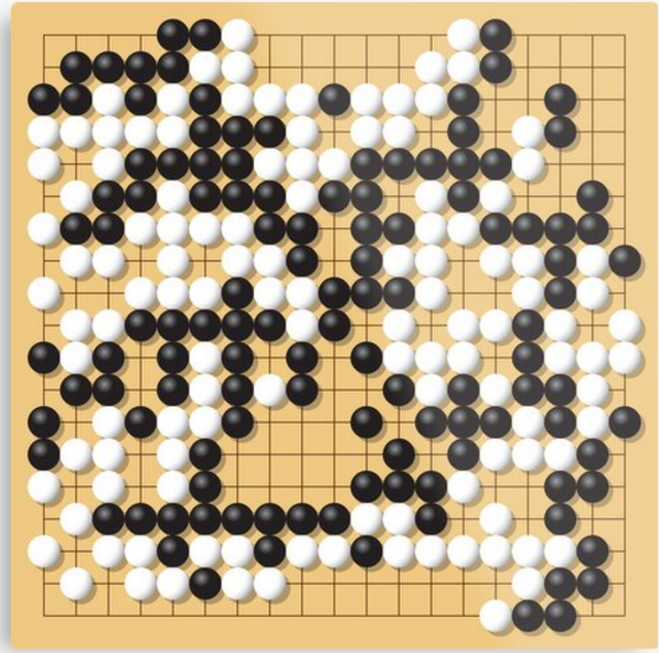
Image credits: redbubble.com

State-Space Complexity Calculation



4

42



172

We need better search strategy instead of exhaustive search

Problem Solving Agent

Goal-Based Rational Agents

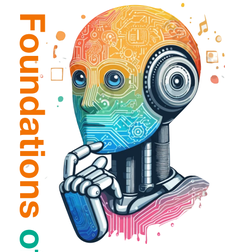
Rational Agent

- **Optimal action** for every possible state
- Optimal action: Which makes the agent most successful
- **Performance measure**: to measure how successful the agent is
- Rational agent has **optimal policy** with its level of knowledge

oestranhomundodek.tumblr

Problem Solving Agent

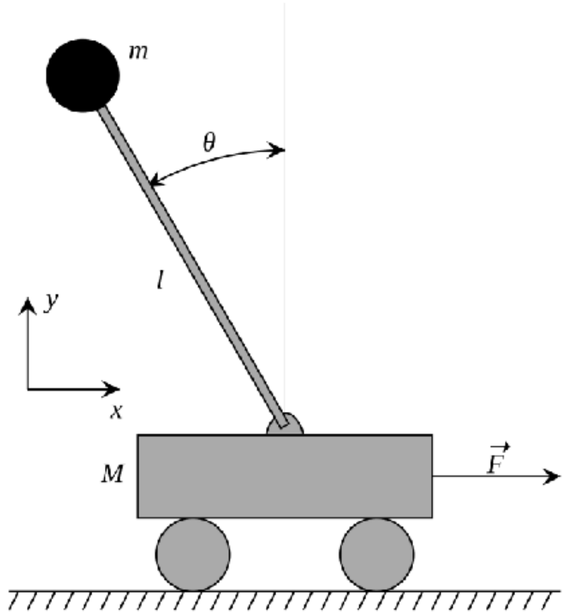
Goal-Based Rational Agents



- Optimal performance
- Find a sequence of actions that satisfy goal

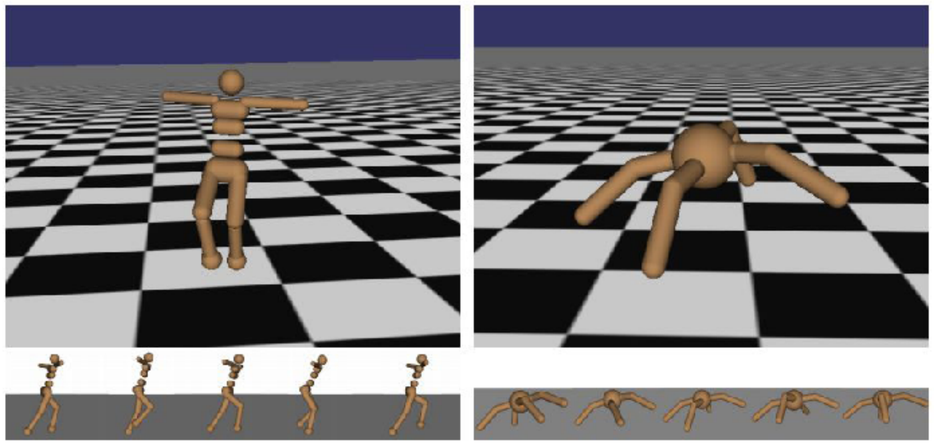
Problem Solving Agent

- Goal: Balance a pole on top of a movable cart
- State: angle, angular speed, position, horizontal velocity
- Action: horizontal force applied on the cart
- Cost/Reward: 1 at each time step if the pole is upright



Robot Locomotion

- Goal: Make the robot move forward
- State: angle and position of the joints
- Action: torque applied on joints
- Cost/Reward: 1 at each time step upward + forward movement



Robot Locomotion

https://www.youtube.com/watch?v=gn4nRCC9TwQ

Self Driving Car

Felix Yu

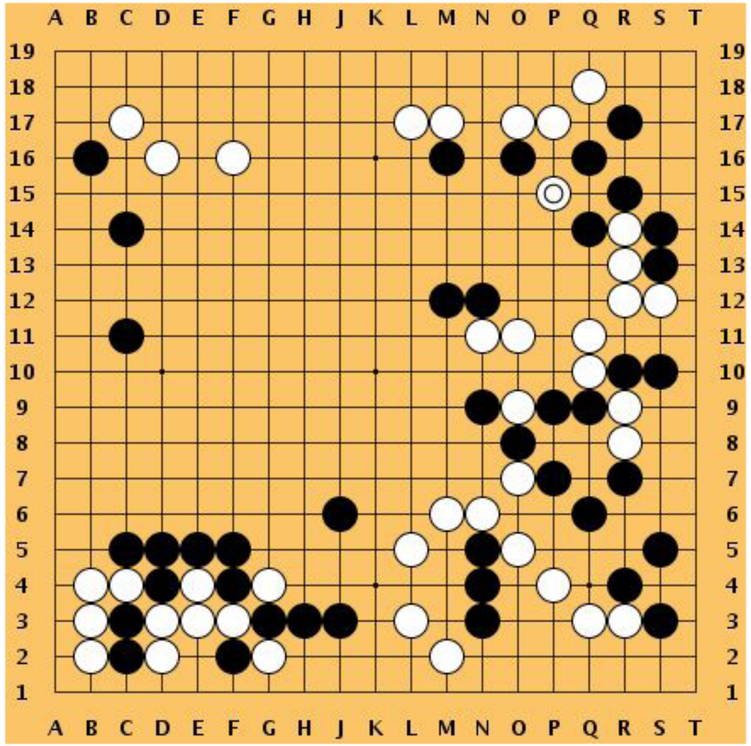
Foundations of AI
AI500

Atari Games

- Goal: Complete game with highest score
- State: Raw pixel input of the game state
- Action: Game control e.g. left, right, up, down
- Cost/Reward: Score increase/decrease at each time point

Go

- Goal: Win the game
- State: Position of all pieces
- Action: where to put the next piece
- Cost/Reward: 1 if win at the end of the game, 0 otherwise



Problem Solving Agents

A **problem** is defined by several items:

- **State Space**: Set of possible states of environment
- **Initial state** of the agent: Starting point
- **Goal States**: End state
- **Actions**: Set of available actions to agent
- **Transition Model**: what each action does
- **Cost function** for every action
- **Path**: A sequence of action
- **Solution**: Path from initial state to Goal State
- **Optimal Solution**: Having lowest path cost

Example: Route Finding Problem

Goal: Shortest route from Arad to Bucharest

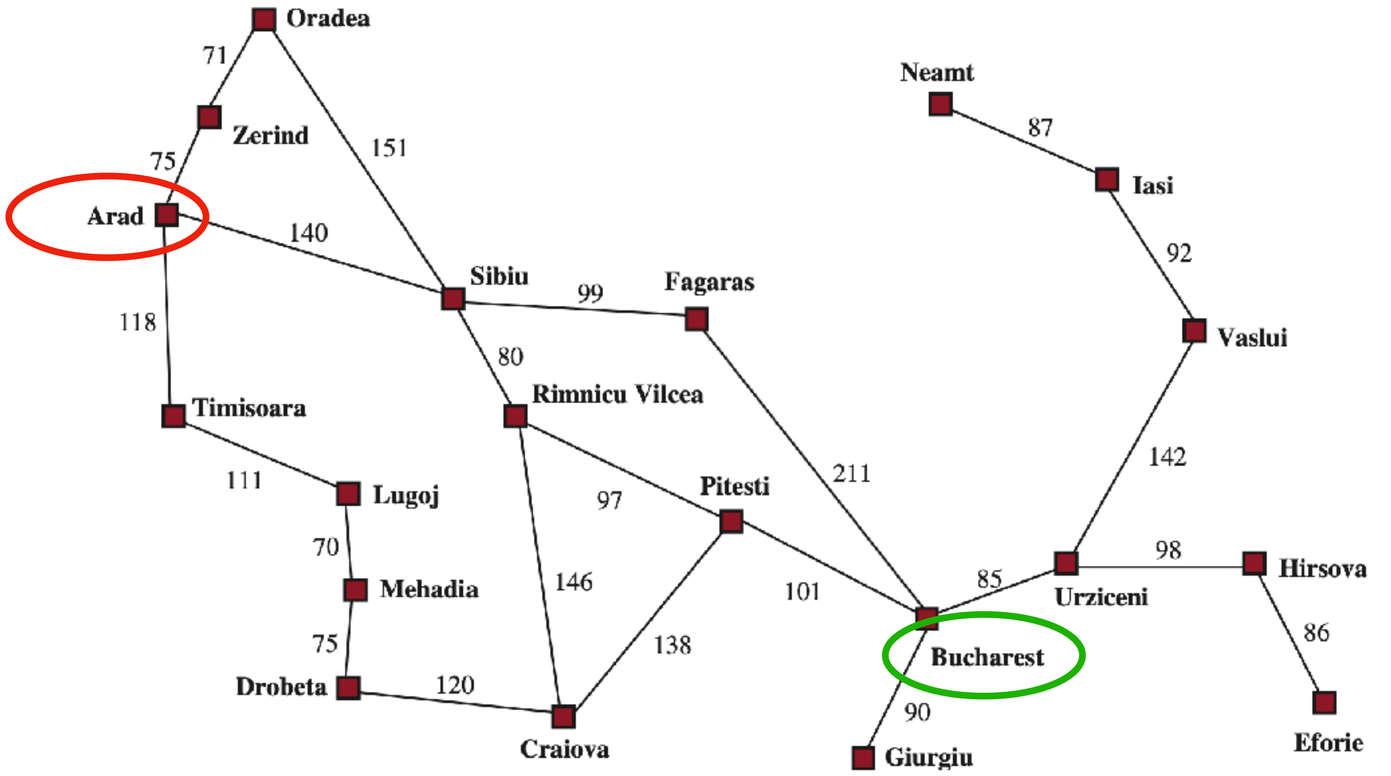


Figure 3.1 A simplified road map of part of Romania, with road distances in miles.

Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Image credits: Stuart Russell & Peter Norvig

Example: Route Finding Problem

Goal: Shortest route from Arad to Bucharest

- **State Space**: Each possible location
- **Initial state** of the agent: ARAD
- **Goal States**: BUCHAREST
- **ACTIONS**: Move from current location to next
- **Transition Model**:
  IF agent_is_in_ARAD
  NEXT_STATE == ZERIND || SIBIU || TIMISOARA
- **Cost function**: A combination of distance + time etc
- **Path**: Move from ARAD to ZERIND to ORADES
- **Solution**: ARAD->ZERIND->ORADES->SIBIU->FAGARAS- >BUCHAREST

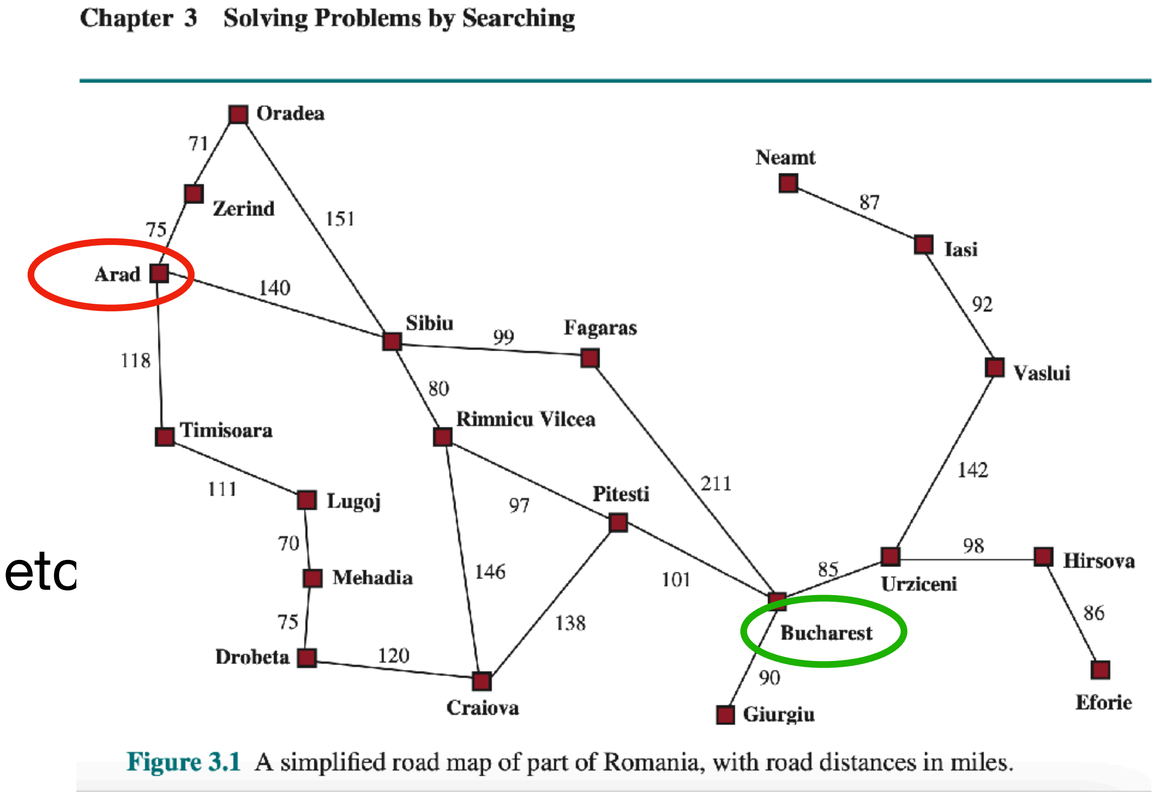


Image credits: Stuart Russell & Peter Norvig

Search Algorithm

Search Algorithms

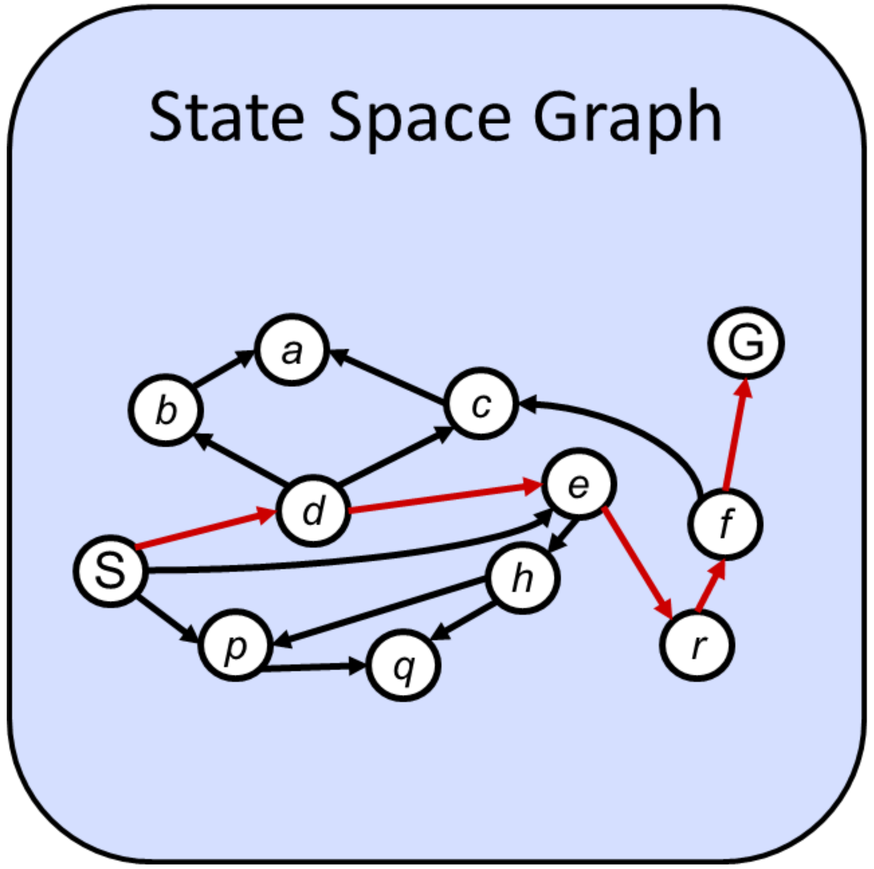
Input

- **State-space graph**: A mathematical representation of a search problem
  - Nodes are (abstracted) world configurations
  - Arcs represent successors (action results)
  - The goal is a set of states/nodes (maybe only one)

Output

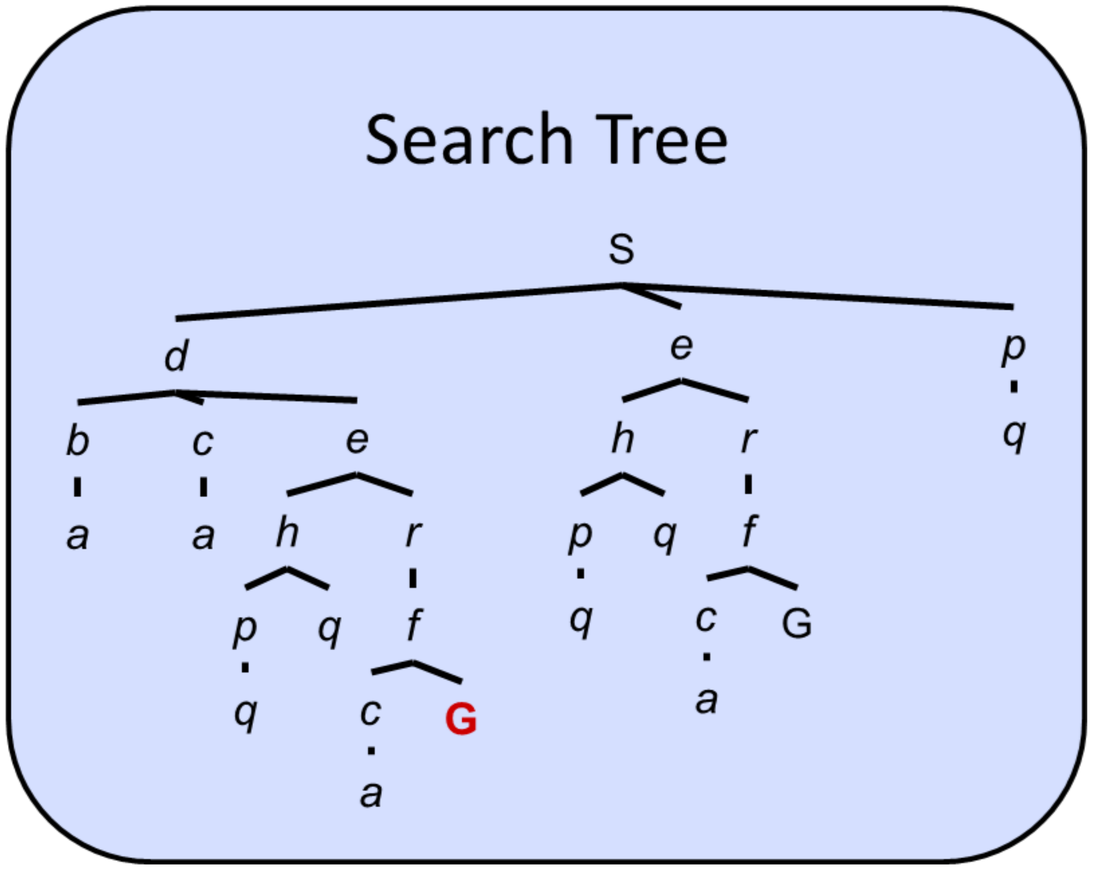
- A **search tree**:
  - A “what if” tree of plans and their outcomes
  - The start state is the root node
  - Children correspond to successors
  - Nodes show states, but correspond to PLANS that achieve those states

Search Algorithms



Each NODE in in the search tree is an entire PATH in the state space graph.

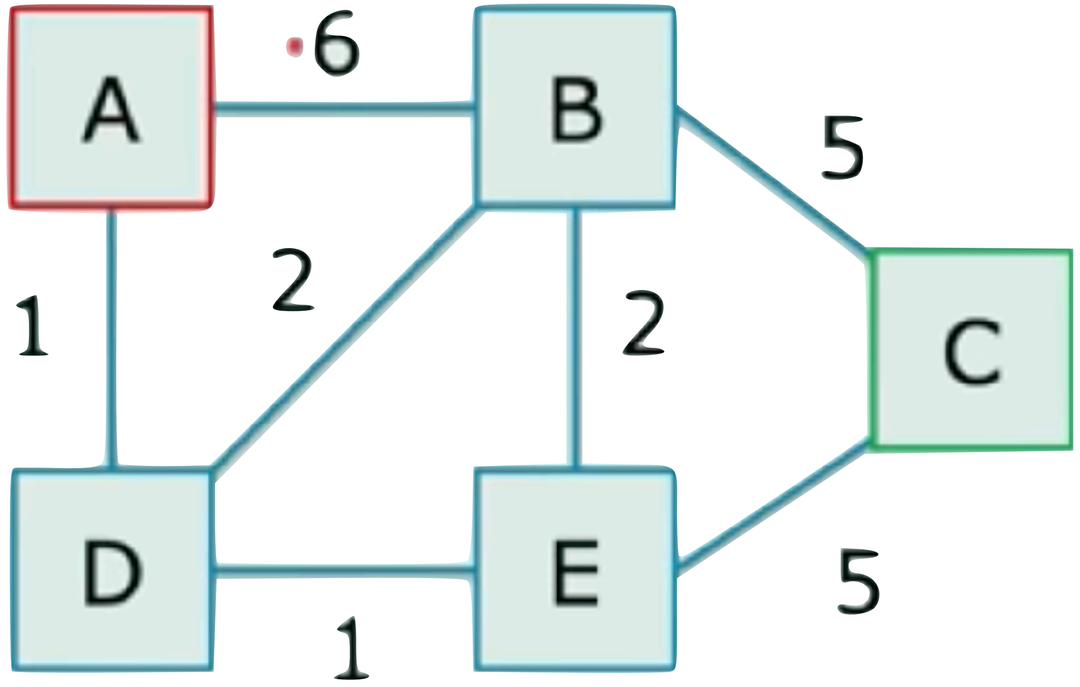
We construct both on demand – and we construct as little as possible.



Search Algorithms

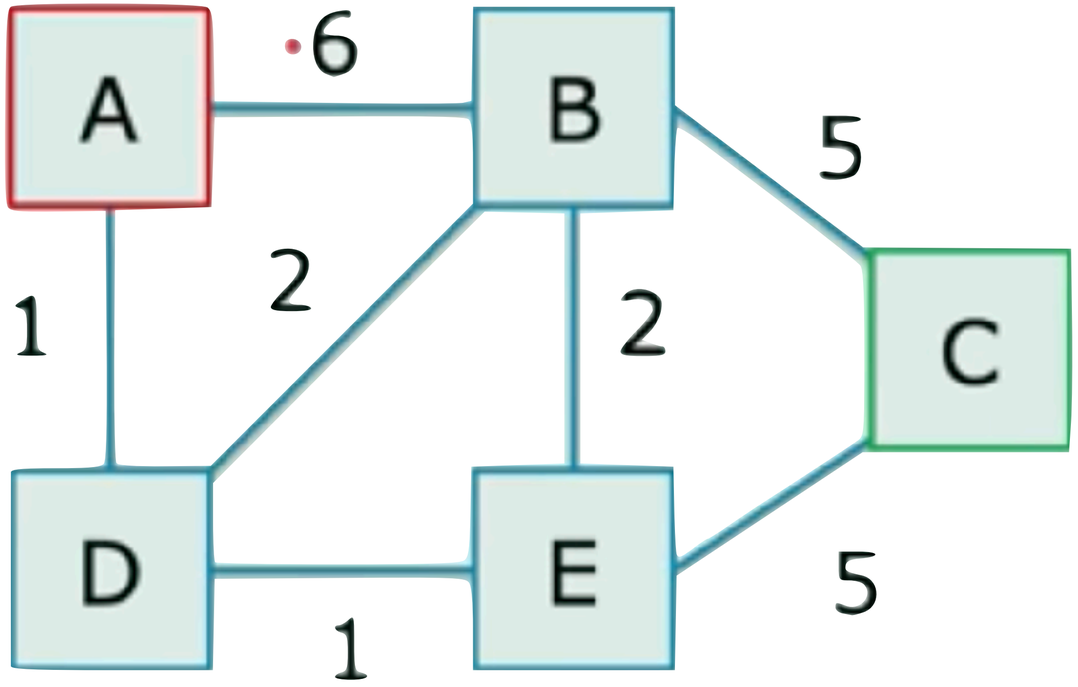
Example

- Initial State
  - A
- Action
  - (A,B) legal if edge
- Goal State
  - C
- Action Cost
  - Edge label (A,B) = 6



Search Algorithms

Example

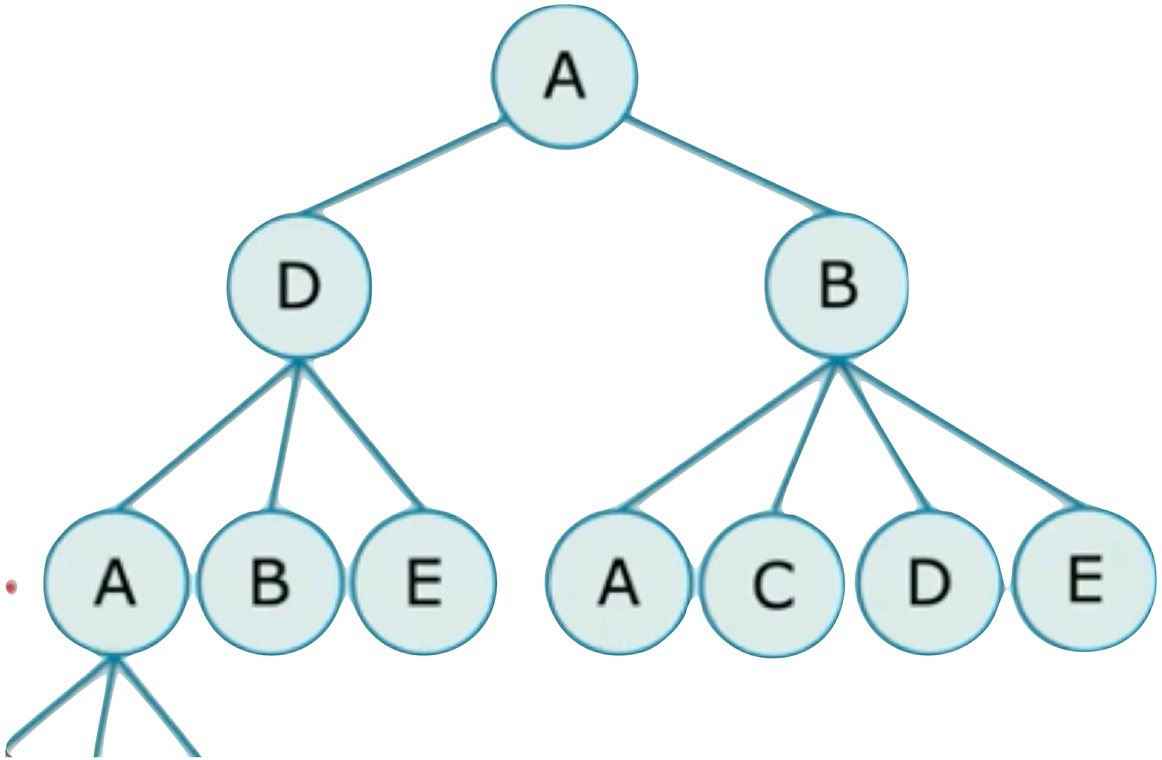


Nodes in Search Tree

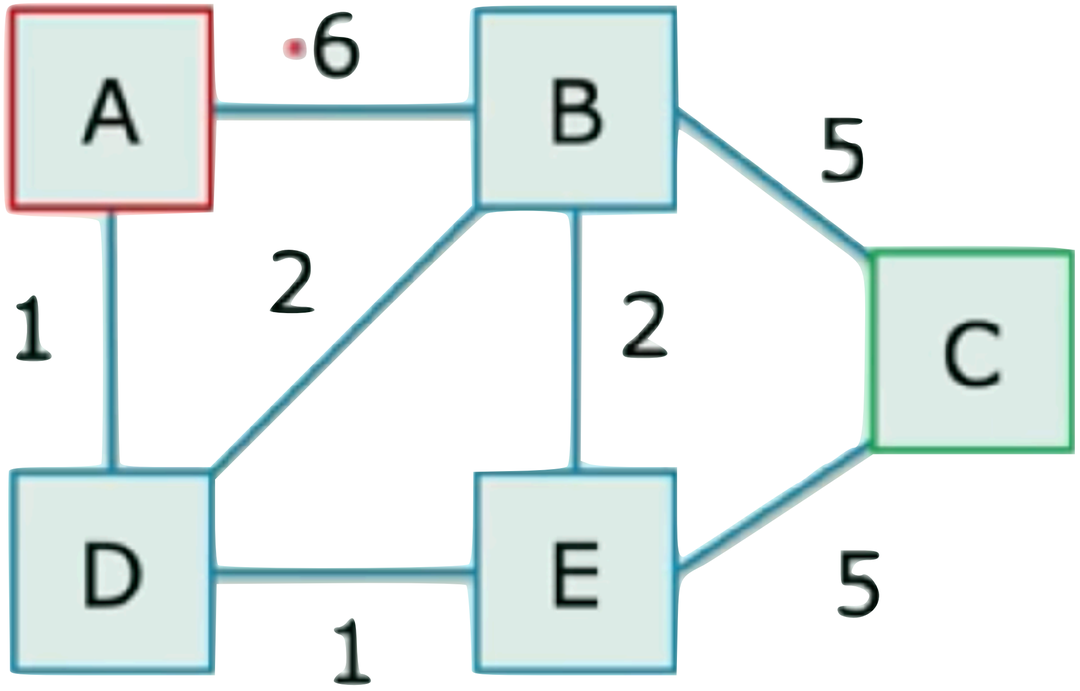
State Environment

Search Algorithms

Example



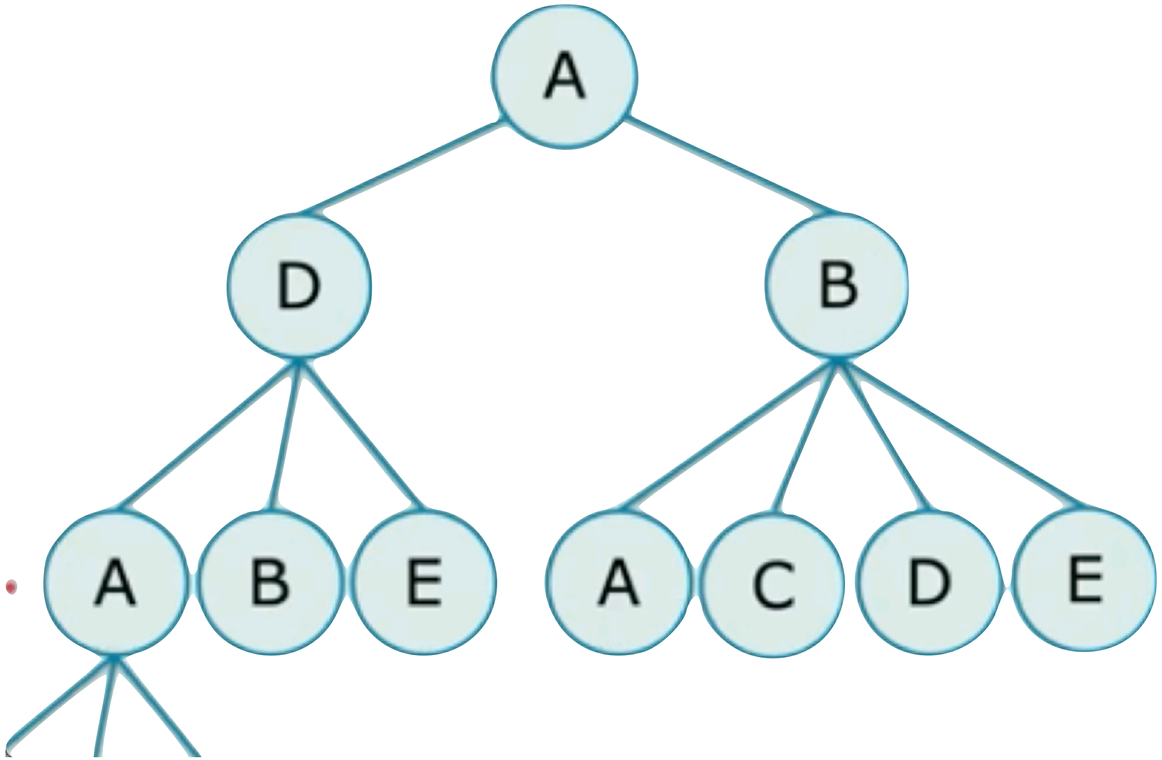
Nodes in Search Tree



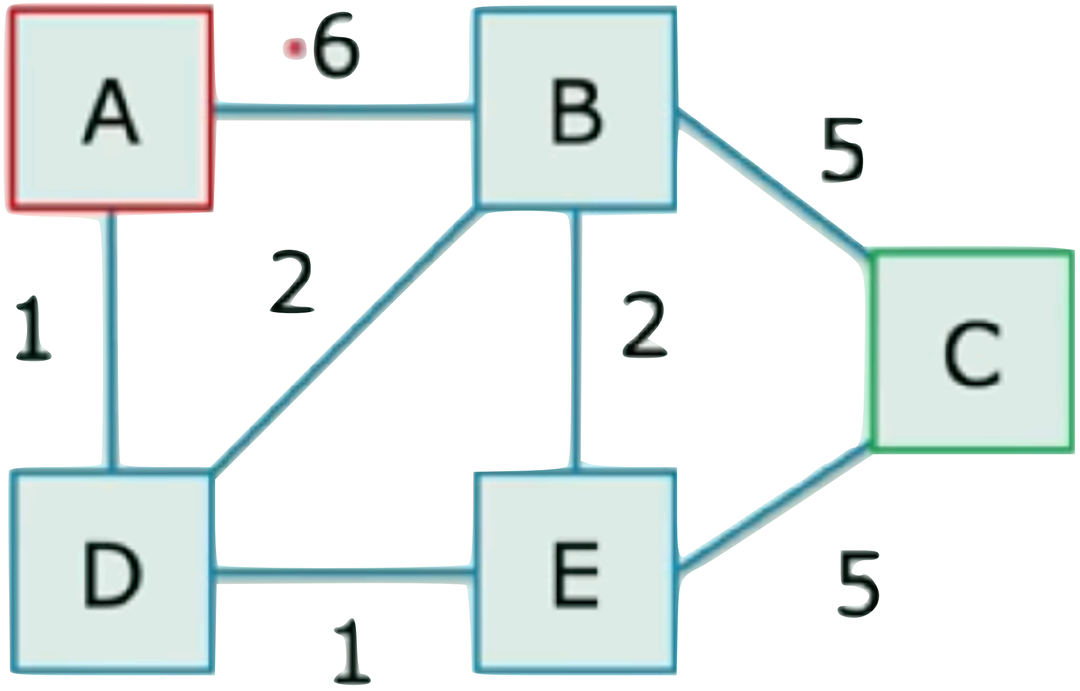
State Environment

Search Algorithms

Example



Nodes in Search Tree
Infinite



State Environment
Finite

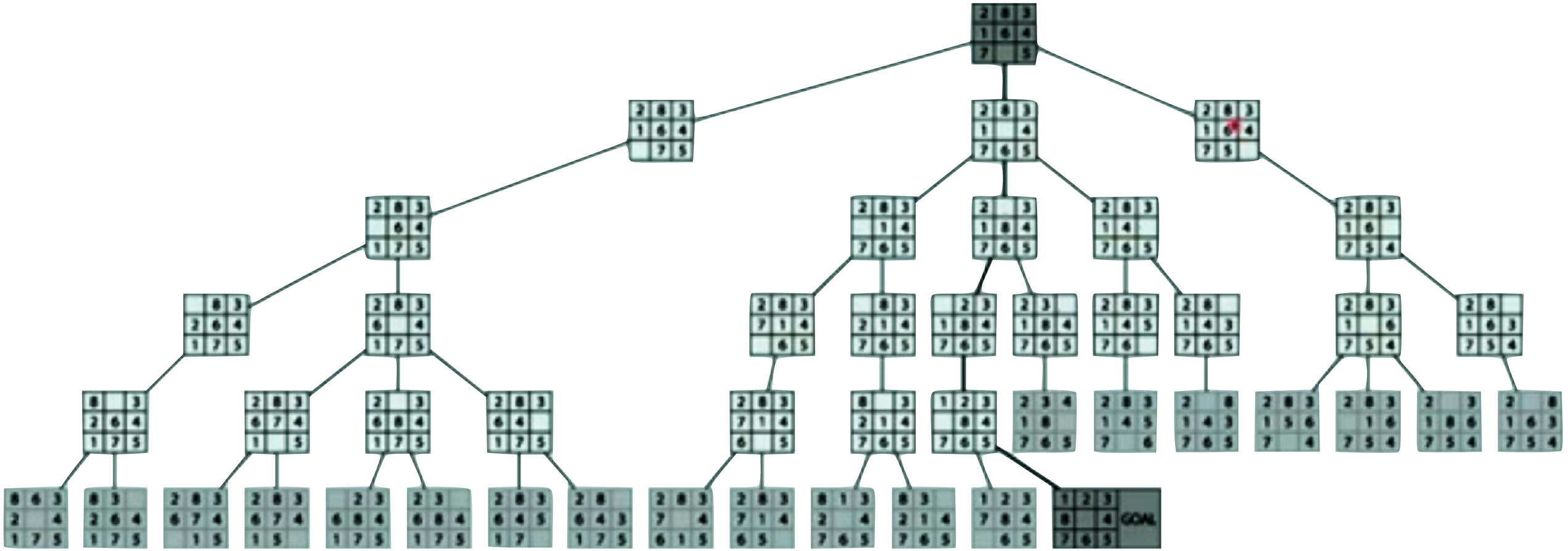
Search Algorithms

8 Puzzle

Foundations of AI
AI500

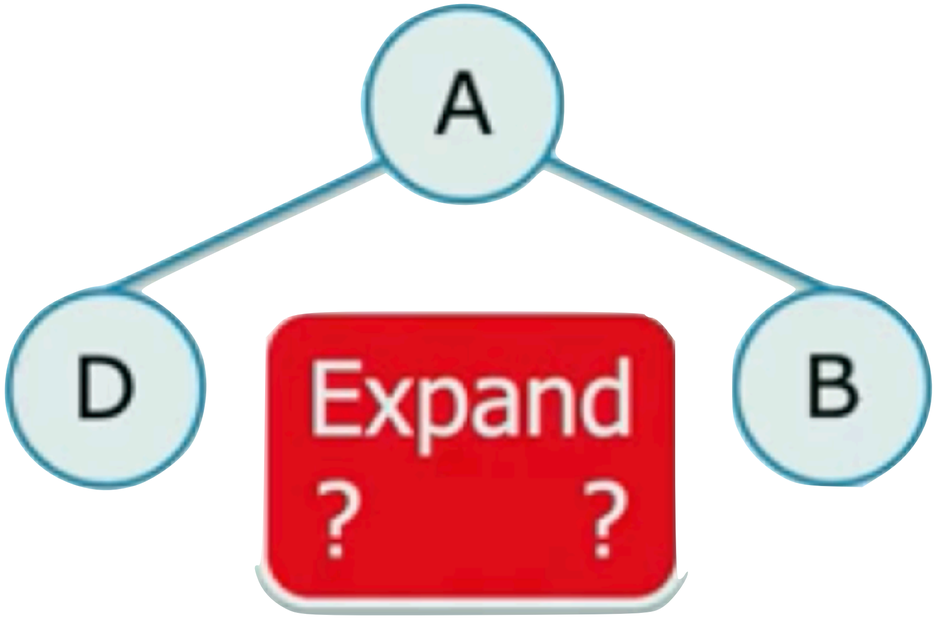
Search Algorithms

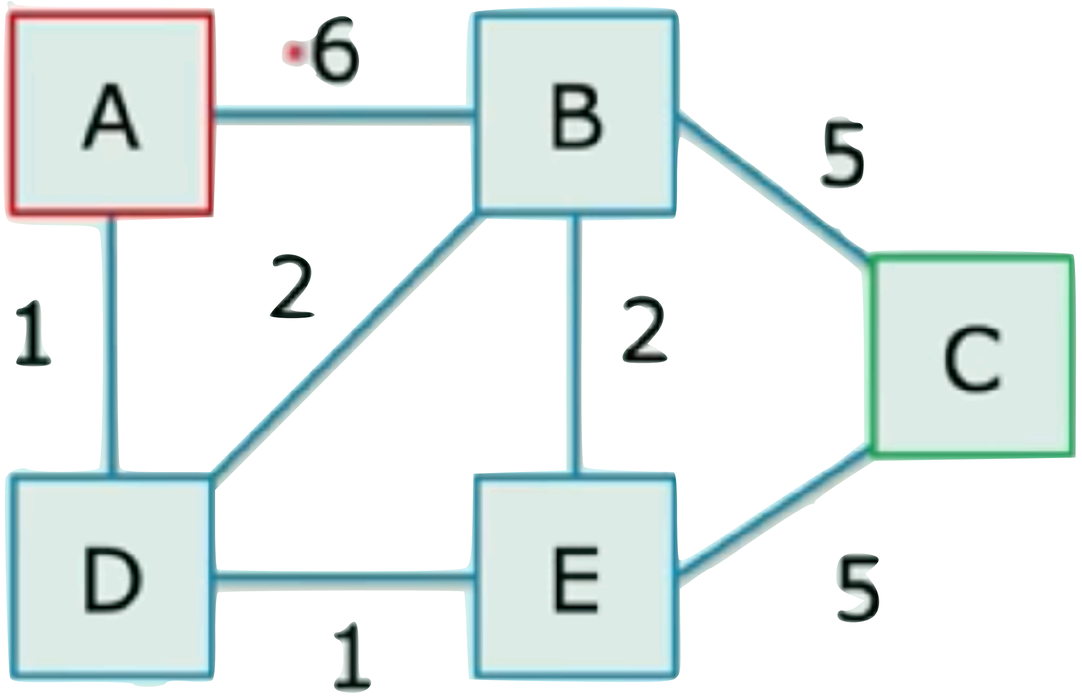
8 Puzzle: Search Tree



Search Algorithms

Which node to expand?





A Generic Search

A Generic Search Algorithm:

- Given a graph, start nodes, and a goal description, incrementally explore paths from the start nodes.
- Maintain a *frontier/Queue* of nodes that are to be explored.
- As search proceeds, the frontier expands into the unexplored nodes until a goal node is encountered.
- The way in which the frontier is expanded defines the search strategy.

.

Generic Tree Search

***Tree search*: Don't check if nodes are visited multiple times**

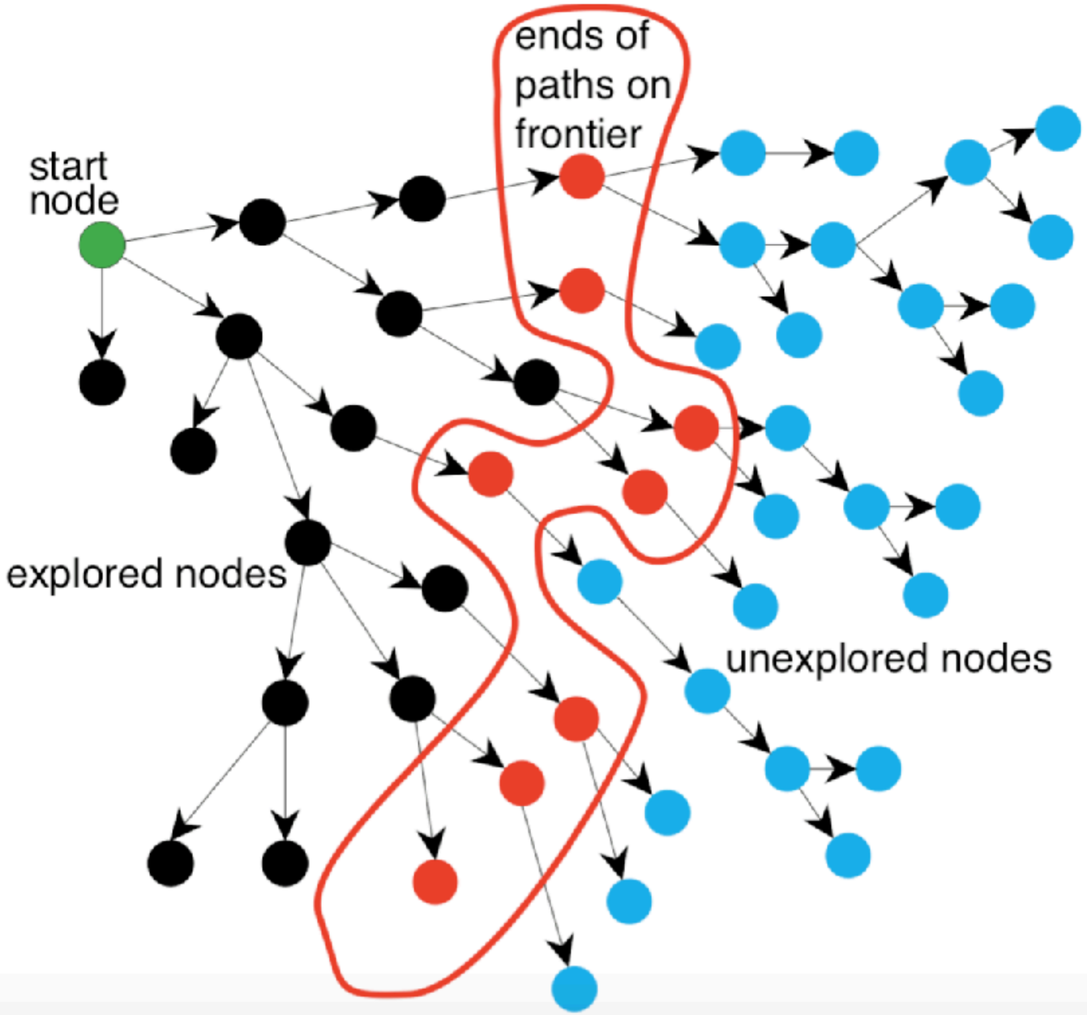


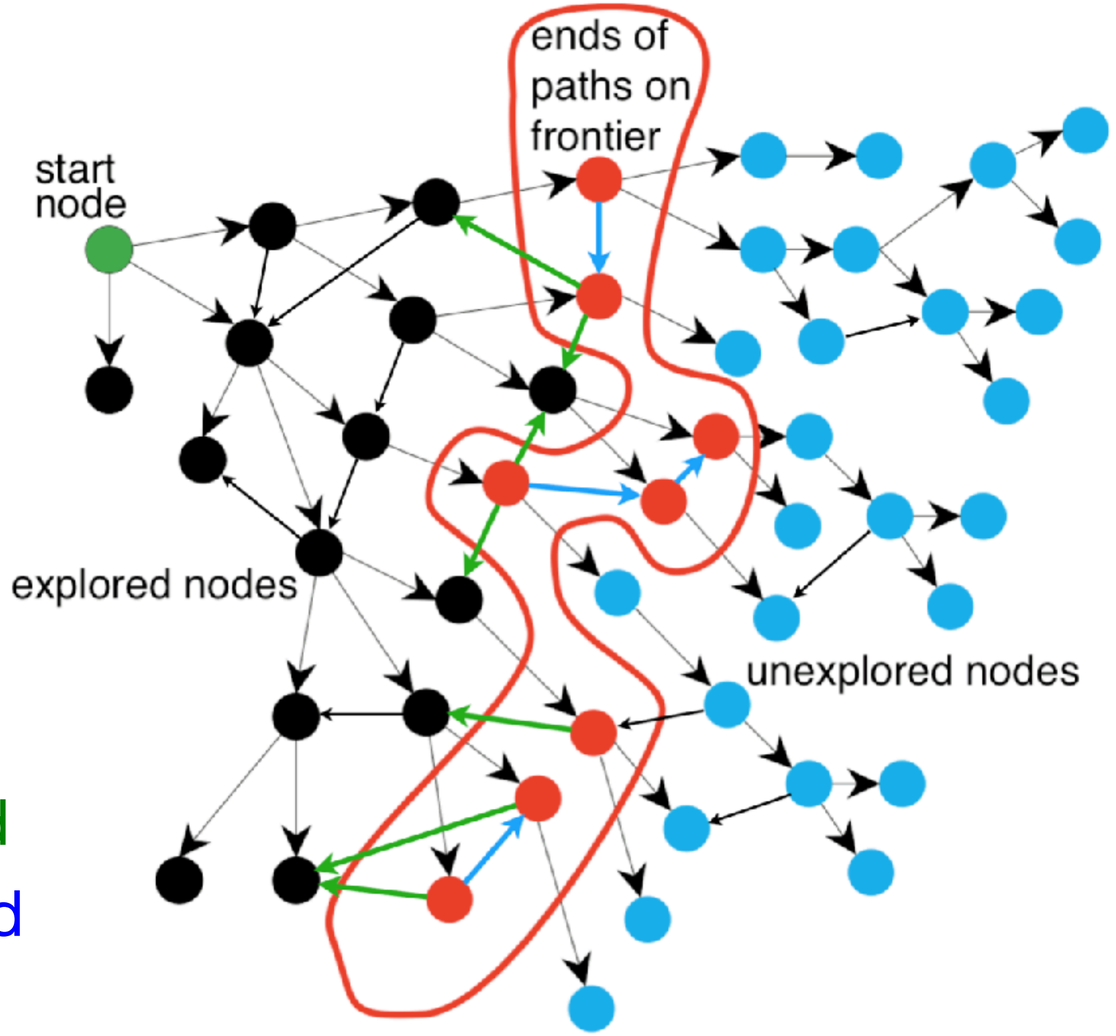
Image Credits: Peter Ljunglöf

A Generic Search Algorithm

Pseudo code

```
function Search(graph, initialState, goalState):
    initialise frontier using the initialState
    while frontier is not empty:
        select and remove node from frontier
        if node.state is a goalState then return solution

        for each child in ExpandChildNodes(node, graph):
            add child to frontier
    return failure
```

Using Tree Search



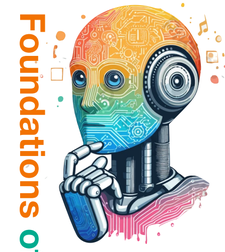
explored nodes might be revisited

frontier nodes might be duplicated

Image Credits: Peter Ljunglöf

Turning Tree Search Into Graph Search

***Graph search*: Keep track of visited nodes**




```
function Search(graph, initialState, goalState):
    initialise frontier using the initialState
    initialise exploredSet to the empty set
    while frontier is not empty:
        select and remove node from frontier
        if node.state is a goalState then return Solution
        add node to exploredSet
        for each child in ExpandChildNodes(node, graph):
            if child is not in frontier or exploredSet
                add child to frontier
    return failure
```

Search Algorithms

Uniformed (Blind)

- Breadth First Search (BFS)
- Depth First Search (DFS)
- Uniform Cost Search (UCS)
- Depth Limited Search (DLS)
- Iterative Deepening Depth First Search (IDDFS)

(Russel & Norvig 3.4)

Informed (Heuristic)

- Greedy Search
- A*

(Russel & Norvig 3.5)

Problem Solving Performance

Problem Solving Performance

- **Completeness**
  - Is it gauranteed to find a solution if it exist?
- **Optimality**
  - Does it find the optimal solution
- **Time Complexity**
  - How long does it take to find a solution
- **Space Complexity**
  - How much memory is needed to run the search?

Time + Space Complexity

- Measured on **size** of input problem/graph
- **Time** measures number of nodes generated
- **Space** measured by how much memory is needed to store the maximum number of nodes

- Search
  - Branching factor b (max sucessor at any node)
  - Depth d (of the shallowest **goal** node)
  - How big is the tree?: Tree size ~ b^d

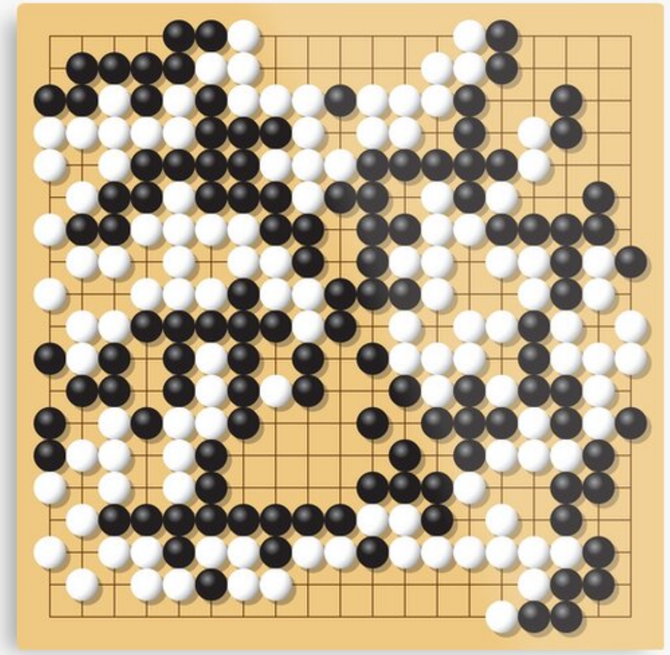
Game-Tree Complexity

Number of possible games that could be played

1	2	3
4	5	6
7	8	9

5

123



360

Uninformed Search

Breadth First Search (BFS)

Depth First Search (DFS)

Uniform Cost Search (UCS)

Depth Limited Search (DLS)

Iterative Deepening Depth First Search (IDDFS)

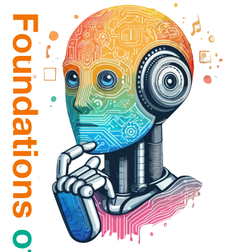
(Russel & Norvig 3.4)

Breadth-First Search (BFS)

Breadth-First Search

- Root node expanded first
- Root successor expanded next
- Their successor expanded next ….
- BFS in General
  - All node at a **given depth** are expanded before any node in the next level

Breadth-First Search



- *Breadth-first search* treats the frontier as a queue.
- It always selects one of the earliest elements added to the frontier.
- If the list of paths on the frontier is $[p_1, p_2, \cdots, p_r]$, then:
  - p_1 is selected (and removed).
  - Its neighbors are added to the end of the queue, after p_r
  - p_2 is selected next.

Illustrative Graph: Breadth-First Search

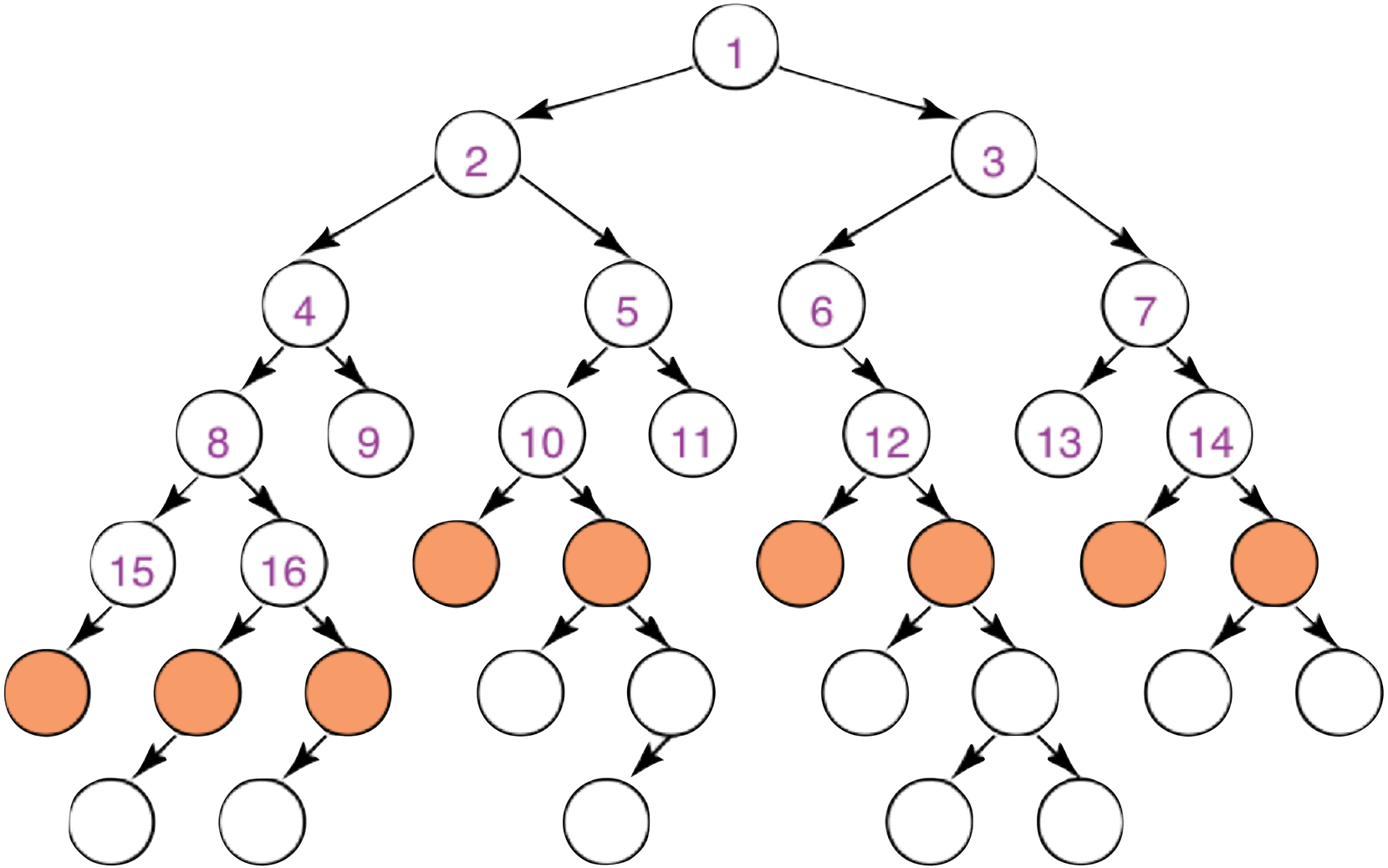
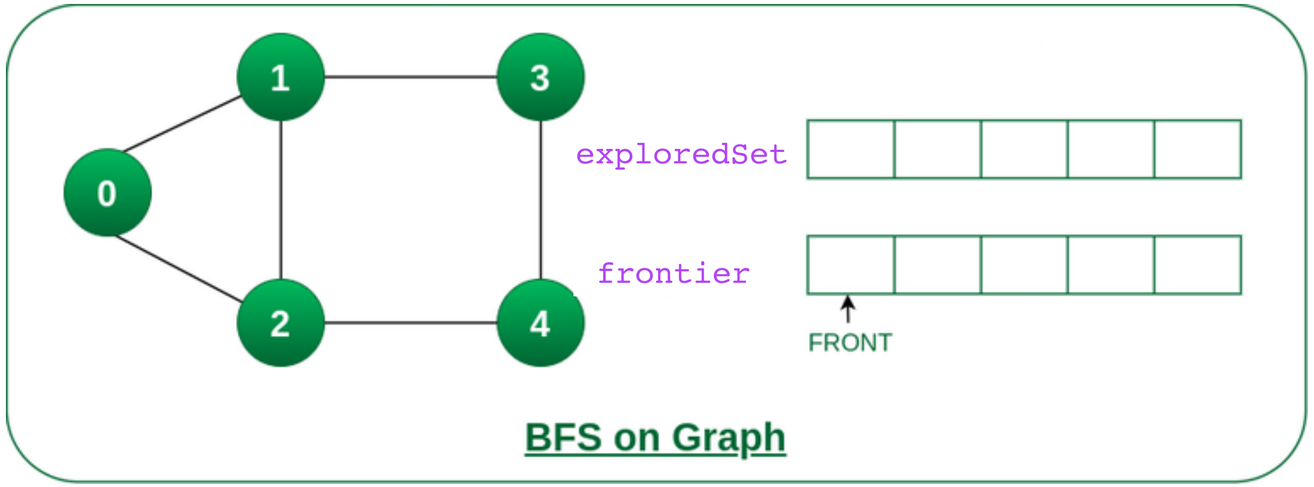


Image Credits: Peter Ljunglöf

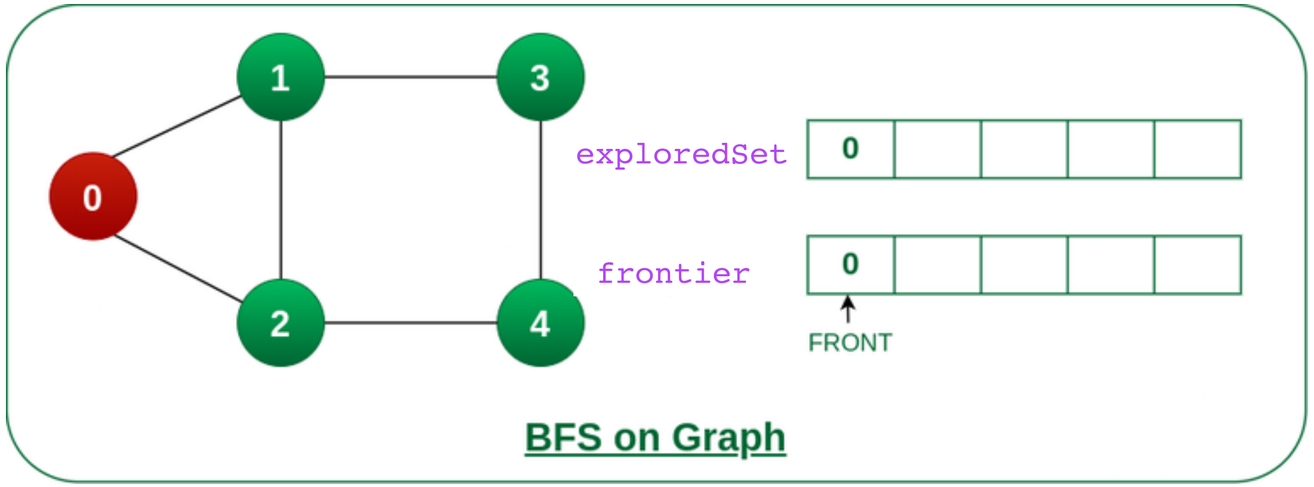
Breadth First Search

Step1: Initially frontier and exploredSet arrays are empty.



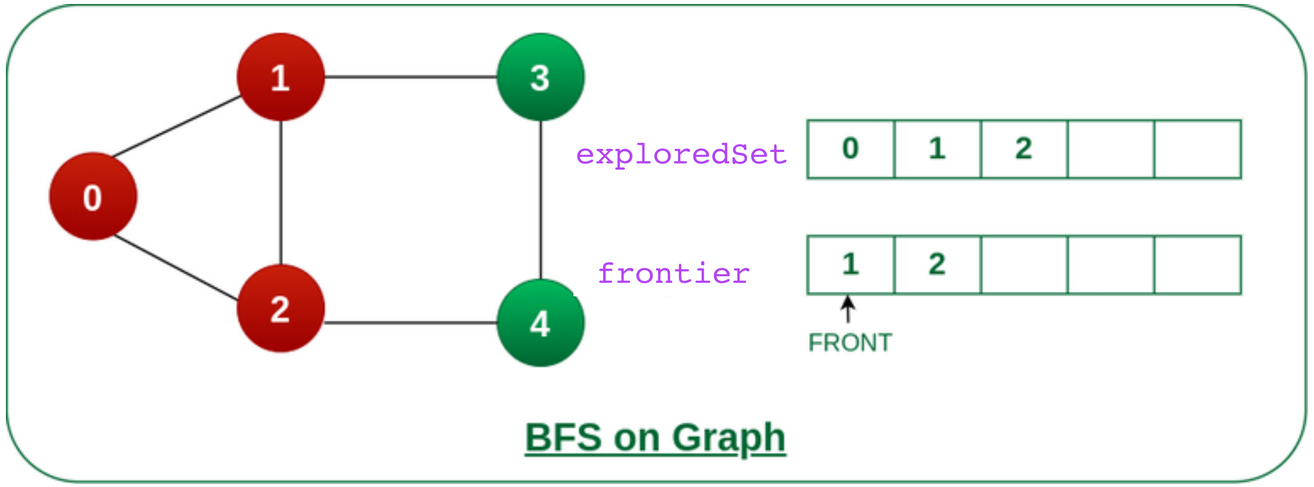
Breadth First Search

Step2: Push 0 into queue and mark it visited



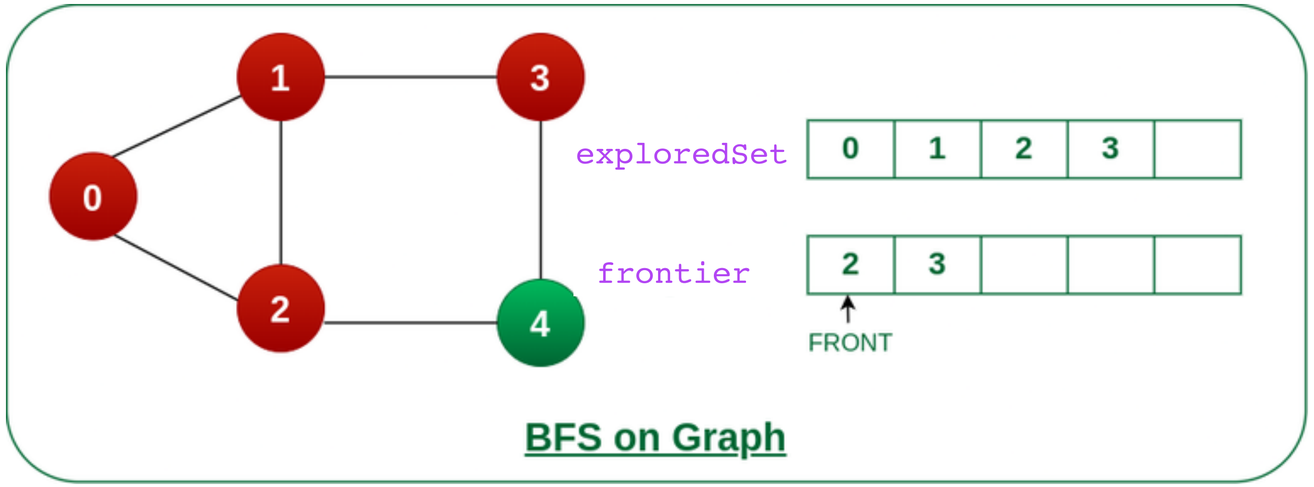
Breadth First Search

Step 3: Remove 0 from the front of queue and visit the unvisited neighbours and push them into queue.



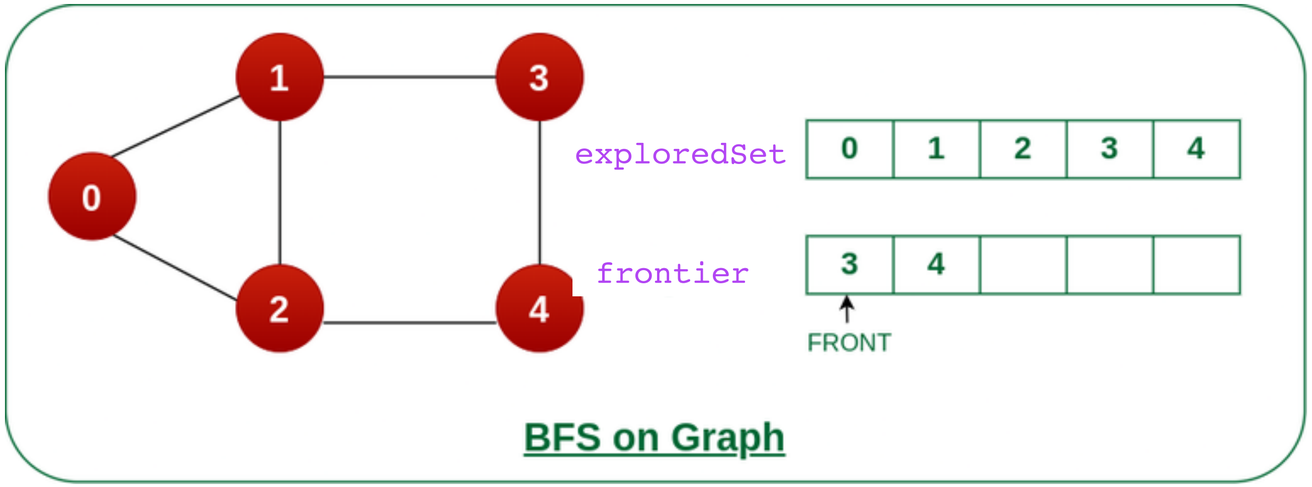
Breadth First Search

Step 4: Remove node 1 from the front of queue and visit the unvisited neighbours and push them into queue.



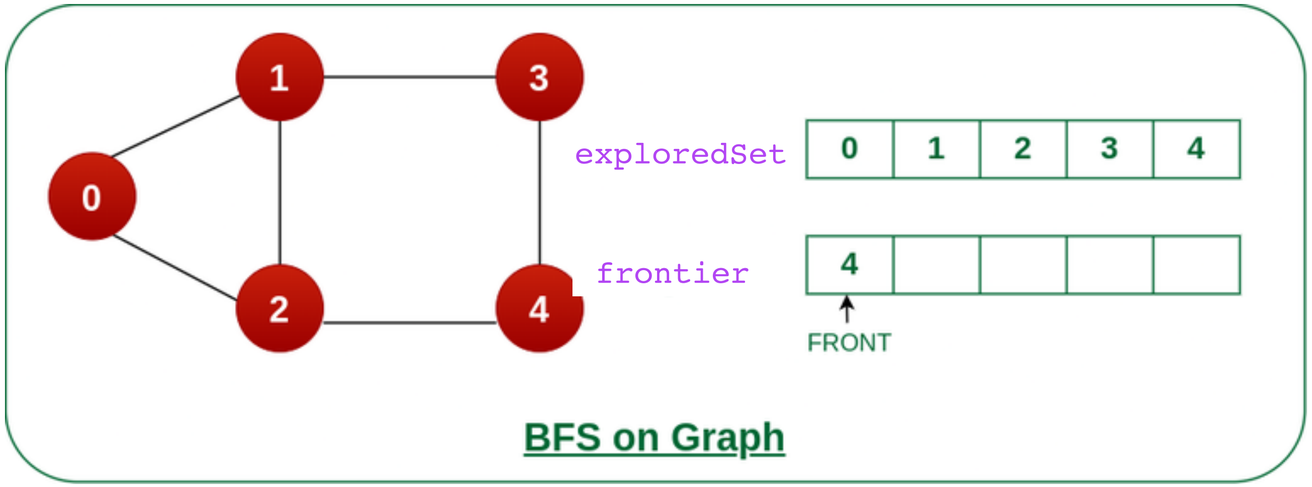
Breadth First Search

Step 5: Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.



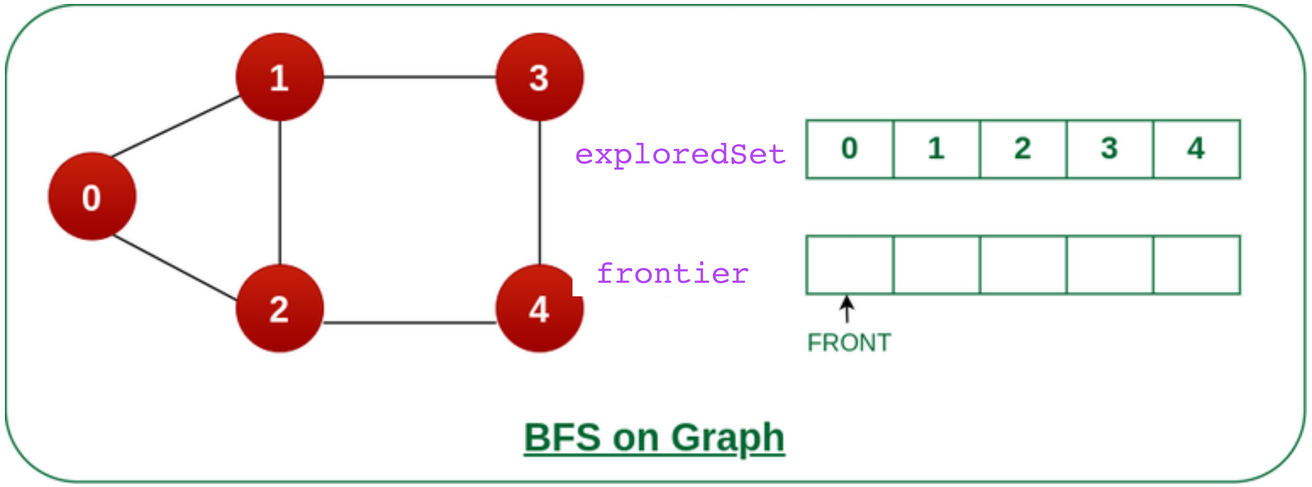
Breadth First Search

Step 6: Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.



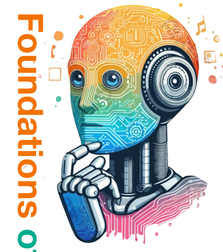
Breadth First Search

Steps 7: Remove node 4 from the front of frontier and visit the unvisited neighbours and push them into frontier.



Breadth First Search

Examples



Breadth-First Search

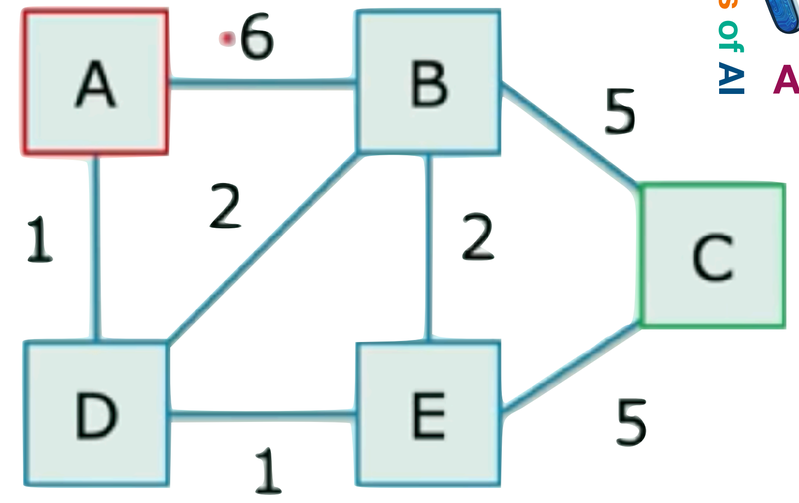
Pseudo code

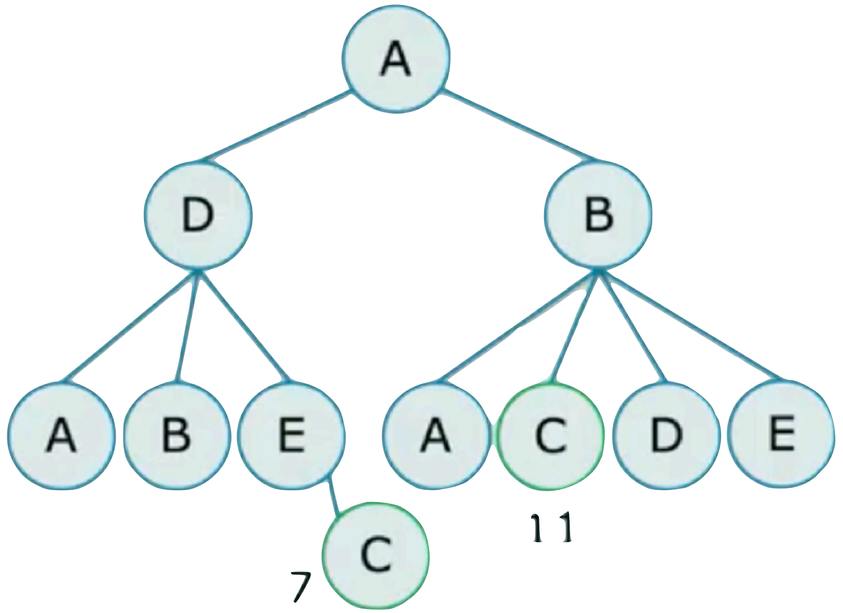
```
function Search(graph, initialState, goalState):
    initialise frontier as Queue using the initialState
    initialise exploredSet to the empty set
    while frontier is not empty:
        select and remove node from frontier
        if node.state is a goalState then return solution
        add node to exploredSet
        for each child in ExpandChildNodes(node, graph):
            if child is not in frontier or exploredSet
                add child to frontier
    return failure
```

Breadth First Search

Problem Solving Performance

- **BFS is Complete**
  - BFS explores entire tree, so it will find a goal node
  - BFS will find the shallowest goal node, since it explores all nodes at a given depth before the nest depth
- **BFS is not Optimal**
  - Shallowest node is not necessarily optimal
  - BFS is optimal if path cost is non-decreasing function on depth
  - BFS is optimal if action cost are all the same

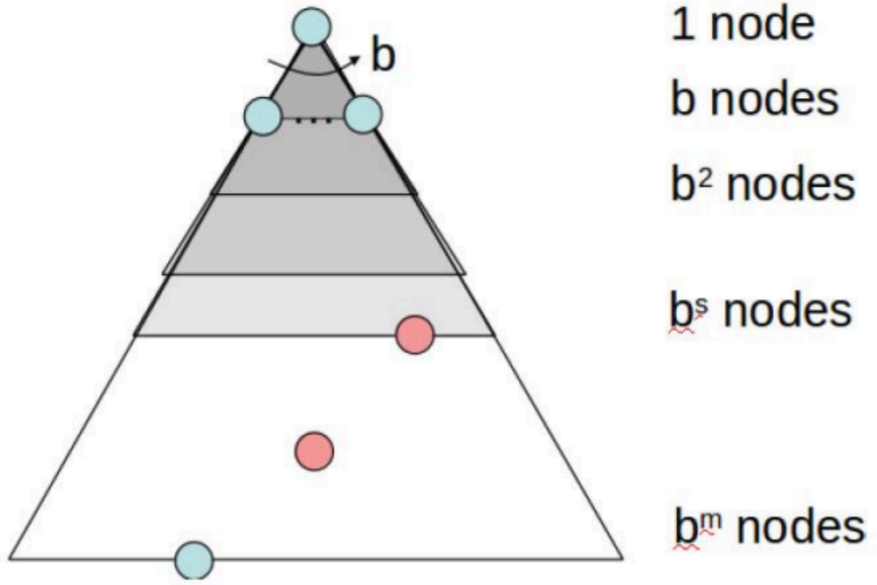




Breadth First Search

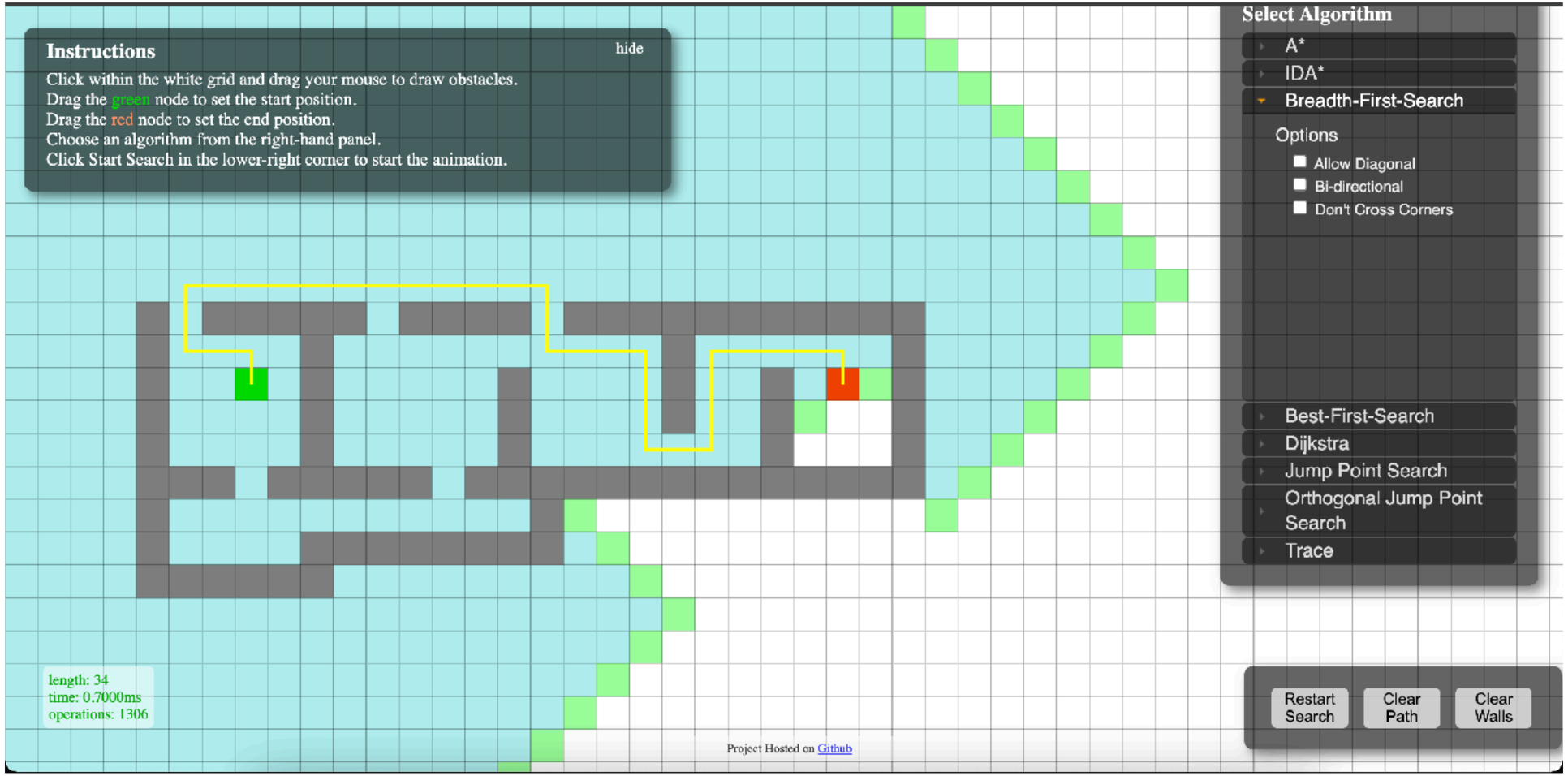
Problem Solving Performance

- **Time Complexity** $O(b^d)$
  - Each state has b children
  - Consider goal at depth d
  - $b \times b \times b \cdots = O(b^d)$ nodes generated
- **Space Complexity** $O(b^d)$
  - Number of nodes in memory = size of frontier



BFS - Demo

https://qiao.github.io/PathFinding.js/visual/



Uniform Cost Search (UCS)

Uniform Cost Search (UCS)

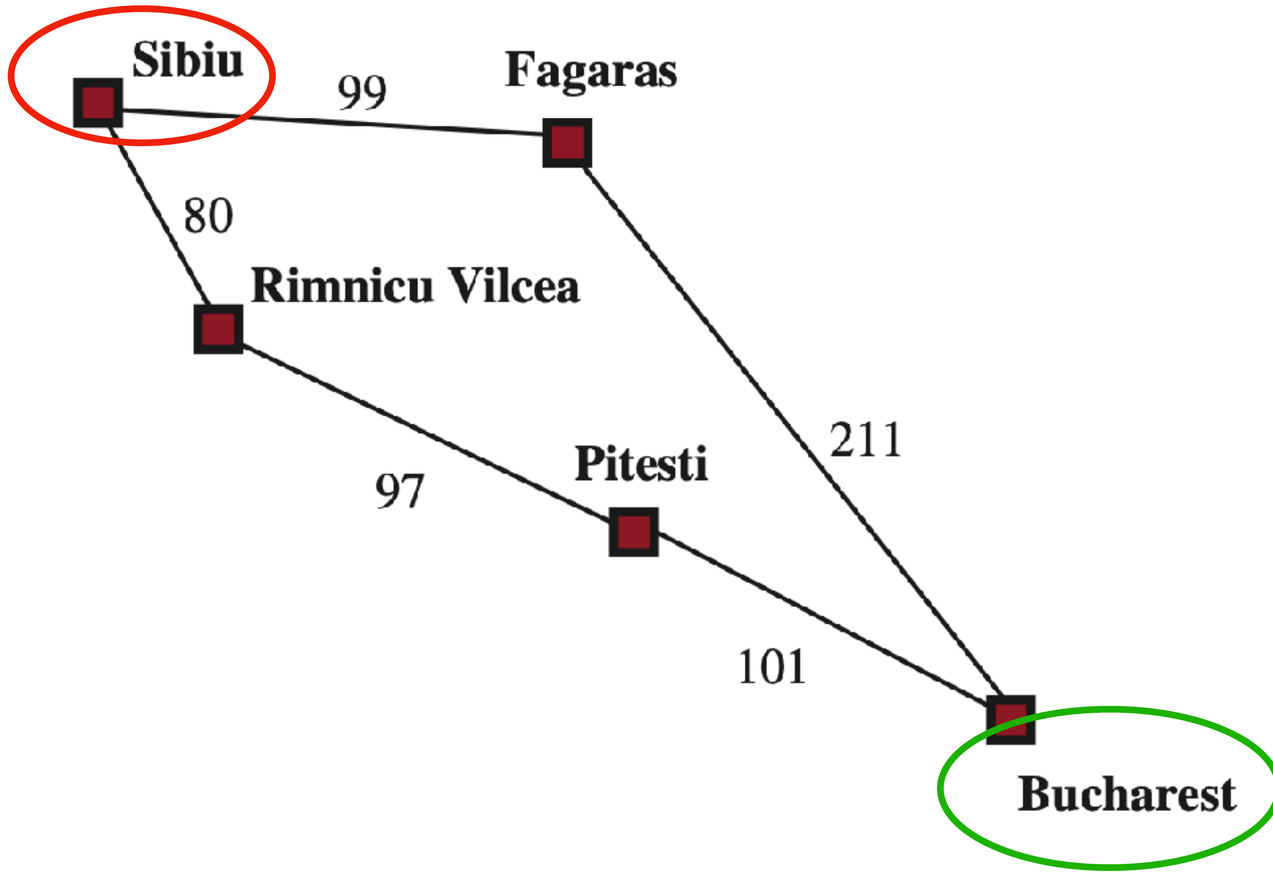
- BFS optimal when all action cost equal because it expands the *shallowest* node
- UCS expands node with *lowest path cost*
- If all costs are equal, equivalent to BFS
- Works only if all costs > 0
- AKA Dijkstra's algorithm

.

Example: UCS

Step 1:

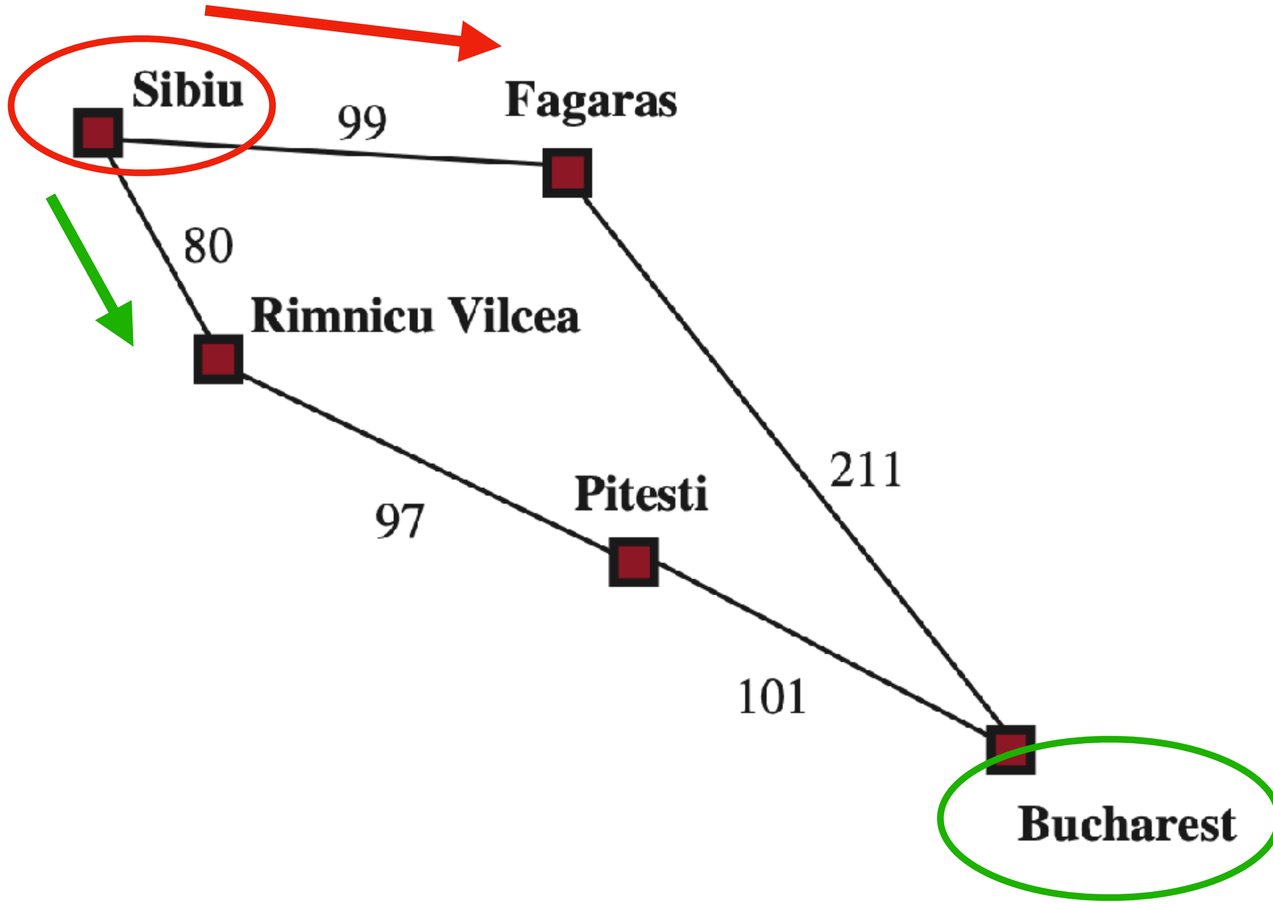
- Visited: Sibiu
- Frontier: Rimnicu Vilcea, Fargaras



Example: UCS

Step 1:

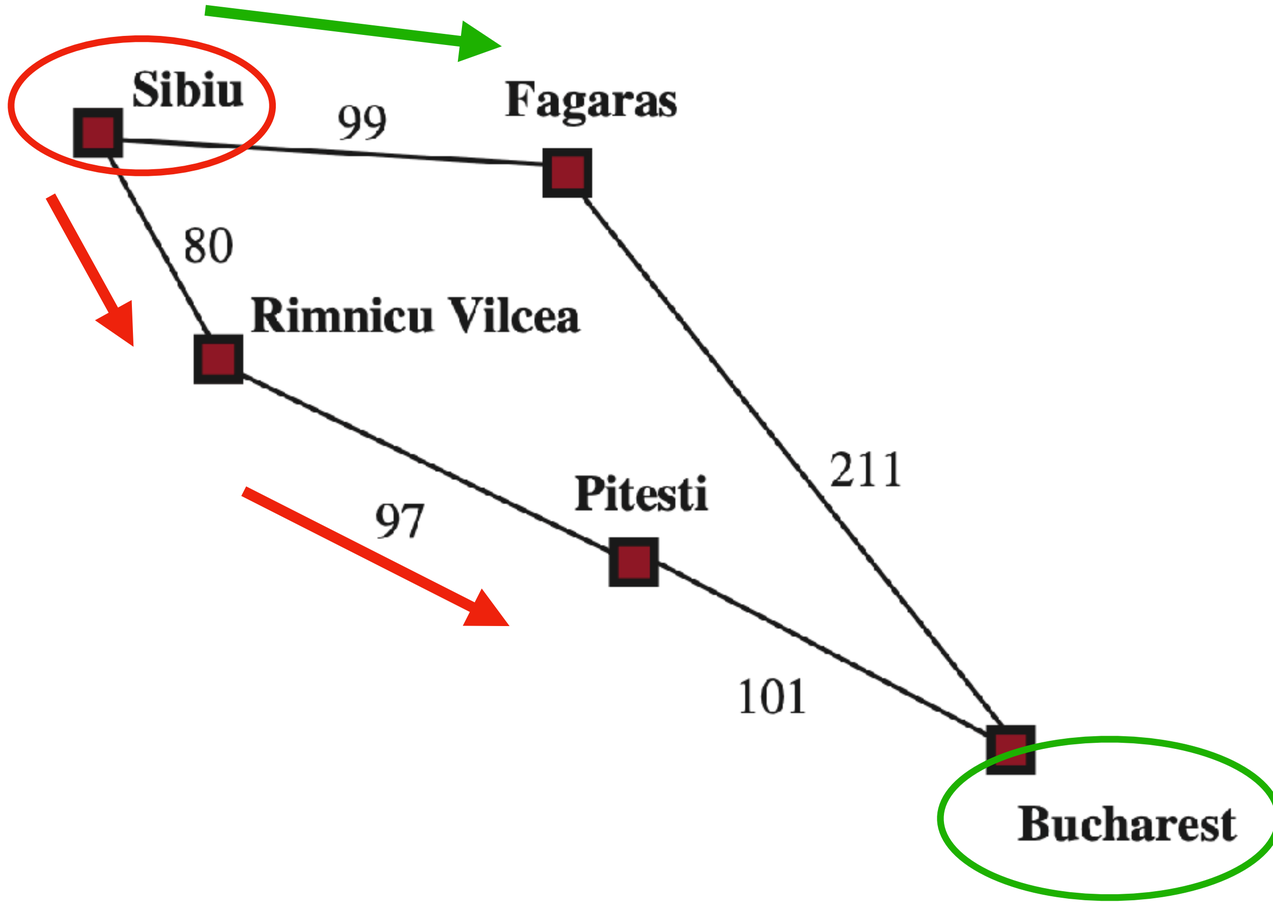
- Visited: Sibiu, Rimnicu Vilcea
- Frontier: Fargaras, Pitesti
- Cost: Sibiu -> Rimnicu Vilcea: 80



Example: UCS

Step 1:

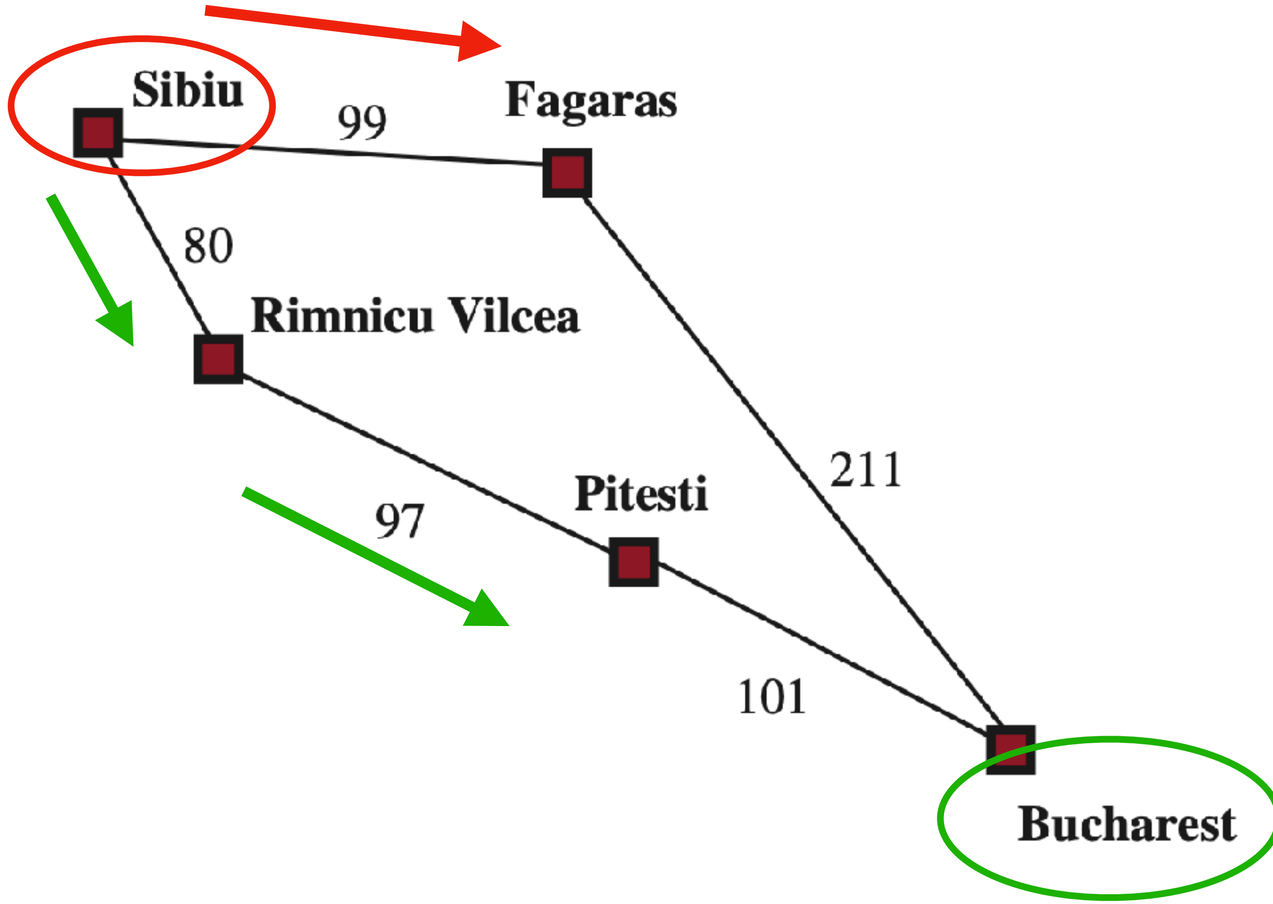
- Visited: Sibiu, Rimnicu Vilcea, Fargaras
- Frontier: Pitesti, Bucharest
- Cost: Sibiu -> Fargaras: 99



Example: UCS

Step 1:

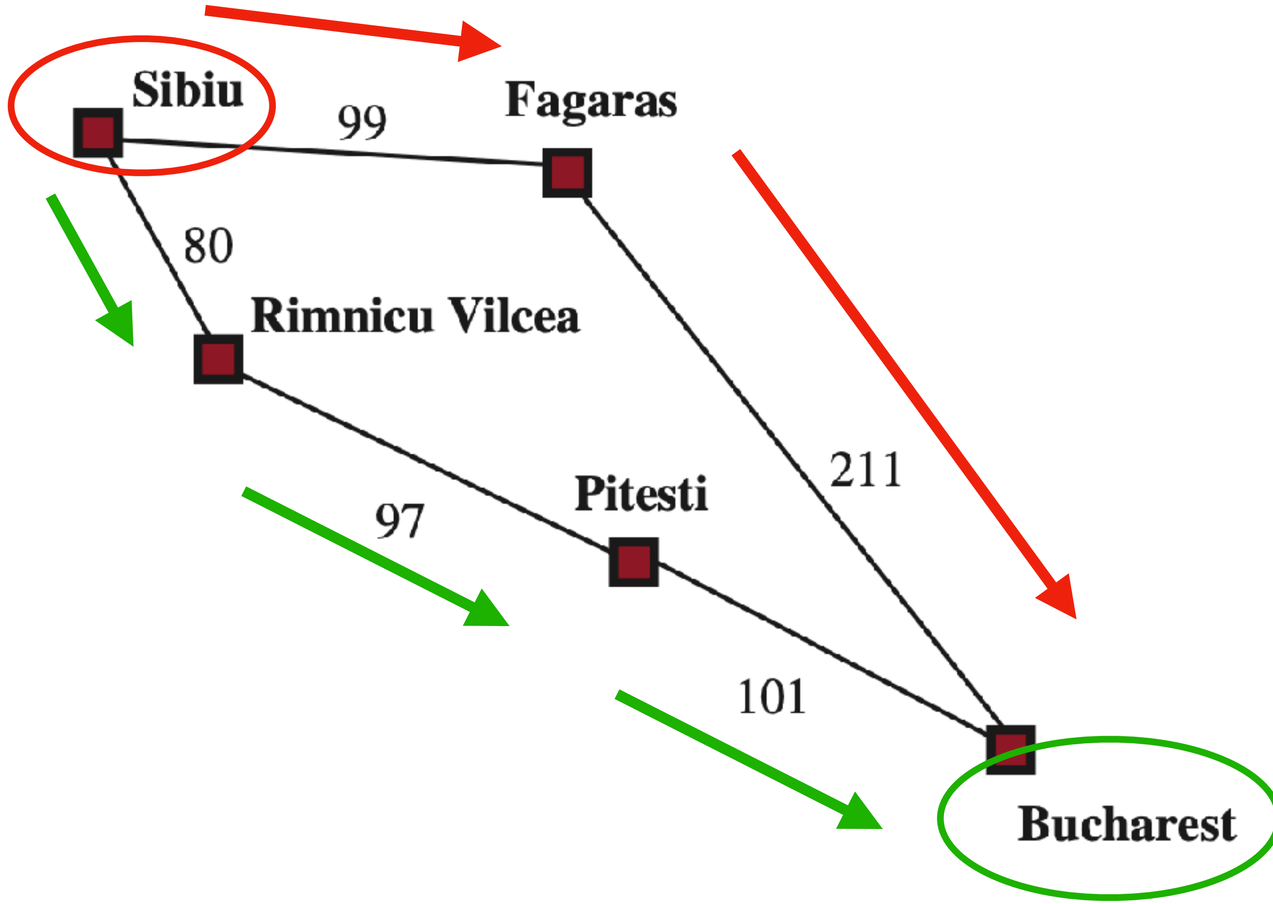
- Visited: Sibiu, Rimnicu Vilcea, Pitesti
- Frontier: Bucharest
- Cost: Sibiu -> Rimnicu Vilcea -> Pitesti: 177



Example: UCS

Step 1:

- Visited: Sibiu, Rimnicu Vilcea, Fagaras, Pitesti
- Frontier: x
- Cost: Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest: 80+97+101 = 278



Uniform-Cost Search

Pseudo code

```
function Search(graph, initialState, goalState):
    initialise frontier using the initialState
    initialise exploredSet to the empty set
    while frontier is not empty:
        select and remove node with minimum path cost from frontier
        if node.state is a goalState then return node
        add node to exploredSet
        for each child in ExpandChildNodes(node, graph):
            if child is not in frontier or exploredSet
                add child to frontier
    return failure
```

Uniform Cost Search

Problem Solving Performance

- UCS is **complete** if b is finite
- UCS is **optimal** if action cost > 0 (ϵ)
- **Time Complexity**
  - Measured by path costs, not depth
  - Can't use b and d for complexity
  - Let C^* be the cost of optimal solution
  - Complexity = $O(b^{1+C^*/\epsilon})$
  - Worst case this can be much bigger than b^d

Depth-First Search (DFS)

Depth-First Search

- *Depth-first search* treats the frontier as a stack.
- It always selects one of the last elements added to the frontier.
- If the list of nodes on the frontier is $[p_1, p_2, p_2, \cdots]$, then:
  - p_1 is selected (and removed).
  - Nodes that extend p_1 are added to the front of the stack (in front of p_2).
  - p_2 is only selected when all nodes from p_1 have been explored.

Illustrative Graph: Depth-First Search

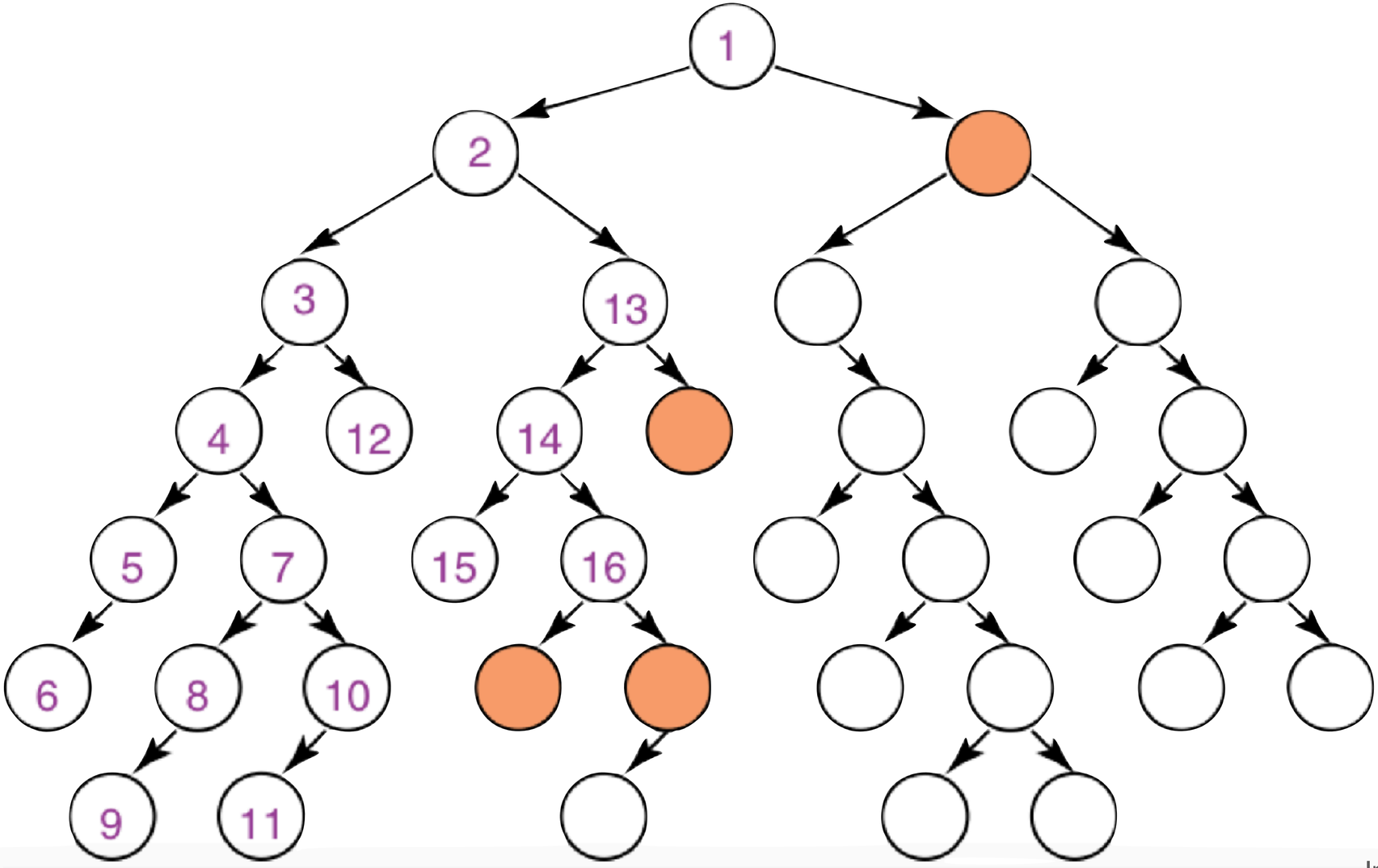


Image Credits: Peter Ljunglöf

Depth-First Search

Pseudo code

```
function Search(graph, initialState, goalState):
    initialise frontier as Stack using the initialState
    initialise exploredSet to the empty set
    while frontier is not empty:
        select and remove node from frontier
        if node.state is a goalState then return node
        add node to exploredSet
        for each child in ExpandChildNodes(node, graph):
            if child is not in frontier or exploredSet
                add child to frontier
    return failure
```

Foundations of AI

AI500

Depth-First Search

Problem Solving Performance

- **DFS is not Complete**
  - In general, DFS is not complete
  - For e.g. can enter infinite loop (enhancement can make it sort of complete)
- **DFS is not Optimal**
  - Returns the first goal found, could be anywhere in the tree, not gauranteed to be optimal
- **Time Complexity** $O(b^m)$, m = max tree depth
  - May go down long possibiliy infinite paths
  - May have to generate entire tree
- **Space Complexity**
  - Only has to store **current path** at any time
  - Use $O(bm)$ memory
  - Stores **successors** of node in the **path** (b factor)